



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ

**ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ**

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

**Ευρωπαϊκό Πορτοφόλι Ψηφιακής Ταυτότητας
(European Digital Identity Wallet) – Μελέτη
Περίπτωσης**

Νικόλαος Καλαμούτσος

Επιβλέπουσα Καθηγήτρια: Ανδριάννα Πρέντζα

ΠΕΙΡΑΙΑΣ

ΙΑΝΟΥΑΡΙΟΣ 2026

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

Ευρωπαϊκό Πορτοφόλι Ψηφιακής Ταυτότητας (European Digital Identity Wallet) – Μελέτη Περίπτωσης

Νικόλαος Καλαμούτσος
A.M.: E21046

ΠΕΡΙΛΗΨΗ

Η παρούσα πτυχιακή εργασία παρουσιάζει την ανάπτυξη μιας εφαρμογής ψηφιακού πορτοφολιού (Digital Wallet) για συσκευές Android, ως μελέτη περίπτωσης του Ευρωπαϊκού Πορτοφολιού Ψηφιακής Ταυτότητας (European Digital Identity Wallet). Η εφαρμογή επιτρέπει τη δημιουργία λογαριασμού, τη σύνδεση μέσω PIN, την προσθήκη και προβολή εγγράφων, καθώς και την παρουσίασή τους για επαλήθευση μέσω κωδικού QR.

Η ανάπτυξη πραγματοποιήθηκε με χρήση της γλώσσας Kotlin και της αρχιτεκτονικής MVVM (Model–View–ViewModel), ενώ για την αποθήκευση δεδομένων χρησιμοποιήθηκε η Room Database. Ιδιαίτερη έμφαση δόθηκε στην ασφάλεια, με μηχανισμούς επαλήθευσης χρήστη και καταγραφής ενεργειών, καθώς και στην πολυγλωσσική υποστήριξη, με μεταφράσεις σε έξι ευρωπαϊκές γλώσσες (Αγγλικά, Ελληνικά, Γερμανικά, Ιταλικά, Ισπανικά και Γαλλικά).

Η εργασία ακολουθεί τις βασικές αρχές της Υλοποίησης Αναφοράς (Reference Implementation) του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας, όπως αυτές καθορίζονται από την Ευρωπαϊκή Επιτροπή, και εφαρμόζει σε απλοποιημένη μορφή τις έννοιες της ιδιωτικότητας, της ασφάλειας και της διαλειτουργικότητας. Τέλος, προτείνονται επεκτάσεις που θα επιτρέψουν την πλήρη συμμόρφωση με το ευρωπαϊκό πρότυπο, όπως η ενσωμάτωση μηχανισμών έκδοσης και παρουσίασης διαπιστευτηρίων, διαχείρισης εμπιστοσύνης και ασφαλούς αποθήκευσης κρυπτογραφικών κλειδίων.

ΘΕΜΑΤΙΚΗ ΠΕΡΙΟΧΗ: Ανάπτυξη Εφαρμογών Android

ΛΕΞΕΙΣ ΚΛΕΙΔΙΑ: Ψηφιακό Πορτοφόλι, Ευρωπαϊκή Ψηφιακή Ταυτότητα, Android, MVVM, Ασφάλεια

ABSTRACT

This thesis presents the design and implementation of a Digital Wallet application for Android, developed as a case study of the European Digital Identity Wallet (EUDI Wallet) initiative. The application allows users to register, log in using a PIN, add and manage personal documents, and present them for validation through QR codes.

It was developed using Kotlin and the MVVM (Model–View–ViewModel) architectural pattern, with data persistence handled by the Room Database. The system emphasizes security via PIN verification, activity logging, and access control and provides multilingual support, offering translations in six European languages (English, Greek, German, Italian, Spanish, and French).

The implementation aligns conceptually with the European Commission's Reference Implementation for the EUDI Wallet, following the same core design principles of privacy, security, and interoperability, while remaining lightweight and fully local. Future work focuses on integrating verifiable credential issuance and presentation, trust management, and secure cryptographic key storage for full compliance with the EUDI architecture.

SUBJECT AREA: Android App Development

KEYWORDS: Digital Wallet, European Digital Identity, Android, MVVM, Security

ΠΕΡΙΕΧΟΜΕΝΑ

ΠΡΟΛΟΓΟΣ.....	11
1. ΕΙΣΑΓΩΓΗ.....	12
1.1 Πλαίσιο και Κίνητρο	12
1.2 Αντικείμενο και Στόχοι της Εργασίας.....	13
1.3 Μεθοδολογία και Τεχνολογίες	13
1.4 Σχέση με την Αναφορά Υλοποίησης (Reference Implementation)	14
1.5 Δομή της Εργασίας.....	14
2. ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ.....	16
2.1 Ανάπτυξη Εφαρμογών Android με Kotlin.....	16
2.2 Το Οικοσύστημα του Android Jetpack.....	16
2.3 Η Αρχιτεκτονική MVVM (Model–View–ViewModel).....	17
2.4 Room Database και ORM προσέγγιση.....	18
2.5 DAO (Data Access Object) Σχεδίαση.....	18
2.6 Repository Pattern.....	19
2.7 ViewModel και LiveData	19
2.8 Ασφάλεια και Διαχείριση Συνεδριών	20
2.9 Συνολική Αρχιτεκτονική και Ροή Δεδομένων.....	20
2.10 Συνοπτική Αξιολόγηση της Αρχιτεκτονικής.....	21
3. ΑΝΑΛΥΣΗ ΚΑΙ ΣΧΕΔΙΑΣΗ ΤΟΥ ΣΥΣΤΗΜΑΤΟΣ	23
3.1 Εισαγωγή	23
3.2 Ανάλυση Απαιτήσεων Συστήματος	23
3.3 Περιγραφή Αρχιτεκτονικής.....	24
3.4 Διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram).....	25
3.5 Δομή και Σχέσεις Δεδομένων – ER Διάγραμμα	26
3.6 UML Διάγραμμα Κλάσεων Βάσης Δεδομένων	27
3.7 Επίπεδα Αρχιτεκτονικής – Layered Diagram.....	28
3.8 Ανάλυση Κύριων Συνιστωσών	29
3.9 Πολυγλωσσική Υλοποίηση.....	29
3.10 Ευθυγράμμιση με το EUDI Reference Implementation.....	30
4. ΕΓΧΕΙΡΙΔΙΟ ΧΡΗΣΗΣ ΤΗΣ ΕΦΑΡΜΟΓΗΣ	31
4.1 Εισαγωγή	31
4.2 Οθόνη Εγγραφής (RegisterFragment).....	31
4.3 Σύνδεση Χρήστη (LoginFragment)	35
4.4 Κεντρική Οθόνη Εγγράφων (DocumentsFragment).....	36
4.5 Προσθήκη Νέου Εγγράφου (AddDocumentFragment).....	38
4.6 Προβολή Εγγράφου (DocumentDetailFragment).....	41

4.7 Αλλαγή PIN (ChangePinFragment)	45
4.7.1 Επαλήθευση Χρήστη	45
4.7.2 Αλλαγή PIN	49
4.8. Πολυγλωσσική Υποστήριξη	50
4.9 Συσχέτιση με το Διάγραμμα Περιπτώσεων Χρήσης	56
4.10 Προειδοποιήσεις και Μηνύματα Εφαρμογής (Warnings and Validation Messages)	57
5. ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΑΙ ΜΕΛΛΟΝΤΙΚΗ ΕΡΓΑΣΙΑ	61
5.1 Εισαγωγή	61
5.2 Συνοπτική Ανακεφαλαίωση της Υλοποίησης	61
5.3 Κύρια Επιτεύγματα και Αποτελέσματα	62
5.3.1 Αρχιτεκτονική και Τεχνολογική Πληρότητα	62
5.3.2 Ασφάλεια και Αξιοπιστία	62
5.3.3 Πολυγλωσσικότητα και Προσβασιμότητα	62
5.3.4 Επεκτασιμότητα και Διαλειτουργικότητα	62
5.3.5 Ευχρηστία και Διεπαφή	62
5.4 Συμπεράσματα	63
5.5 Μελλοντική Εργασία	63
5.5.1 Υποστήριξη Verifiable Credentials	63
5.5.2 Trust Frameworks και Ευρωπαϊκές Λίστες Εμπιστοσύνης	64
5.5.3 Προηγμένοι Μηχανισμοί Ασφάλειας	64
5.5.4 Ενσωμάτωση Cloud Services με Κρυπτογράφηση	64
5.5.5 Πιστοποίηση και Συμμόρφωση με EUDI Sandbox	64
5.5.6 Ανάλυση Χρηστικότητας και Απόκρισης	64
5.5.7 Πραγματική Επαλήθευση μέσω SMS	64
5.6 Συνολική Εκτίμηση και Συνεισφορά	65
ΠΙΝΑΚΑΣ ΟΡΟΛΟΓΙΑΣ	66
ΣΥΝΤΜΗΣΕΙΣ – ΑΡΚΤΙΚΟΛΕΞΑ – ΑΚΡΩΝΥΜΙΑ	67
ΠΑΡΑΡΤΗΜΑ Ι – ΚΩΔΙΚΑΣ ΕΦΑΡΜΟΓΗΣ	68
1. Εισαγωγή	68
2. Επίπεδο Παρουσίασης (UI Elements – Fragments)	68
3. Επίπεδο ViewModel	69
4. Επίπεδο Repository	69
5. Επίπεδο Βάσης Δεδομένων (Room)	70
6. Υποστηρικτικές Κλάσεις	73
7. Adapter	74
ΠΑΡΑΡΤΗΜΑ ΙΙ – ΔΙΑΓΡΑΜΜΑ ΠΛΟΗΓΗΣΗΣ & ΠΟΛΥΓΛΩΣΣΙΑ	75

1. Διάγραμμα Πλοήγησης (Navigation Graph).....	75
2. Πολυγλωσσική Υποστήριξη.....	76
2.1 Ενδεικτικές Συμβολοσειρές (Translations Example).....	76
2.2 Πίνακας Κρατών-Μελών Ε.Ε. (Απόσπασμα).....	76
ΒΙΒΛΙΟΓΡΑΦΙΚΕΣ ΑΝΑΦΟΡΕΣ	77

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ (Diagrams/Graphs)

Διάγραμμα 2.3.1: Αρχιτεκτονική MVVM [7]	17
Διάγραμμα 2.9.1: Αρχιτεκτονική και Ροή Δεδομένων	21
Διάγραμμα 3.4.1: UML Use Case Diagram [2]	25
Διάγραμμα 3.5.1: ER Diagram	26
Διάγραμμα 3.6.1: UML Database Class Diagram [2]	27
Διάγραμμα 3.7.1: UML Layered Class Diagram [2]	28
Διάγραμμα παραρτήματος: fragment navigation graph	75

ΚΑΤΑΛΟΓΟΣ ΕΙΚΟΝΩΝ (Figures)

Εικόνα 4.2.1 : Οθόνη Εγγραφής (RegisterFragment)	31
Εικόνα 4.2.2 : Αποστολή Επαλήθευσης Email	32
Εικόνα 4.2.3 a : Μήνυμα Επαλήθευσης Email.....	33
Εικόνα 4.2.3 b : Πίνακας Χρηστών Firebase (Email Authentication table)	33
Εικόνα 4.2.4 : Σελίδα Επιβεβαίωσης	33
Εικόνα 4.2.5 : Λήψη Κωδικού SMS	34
Εικόνα 4.2.6 : Επαλήθευση Τηλεφώνου.....	34
Εικόνα 4.2.7 : Επιτυχής Ολοκλήρωση Εγγραφής και Μετάβαση στη Σύνδεση.....	35
Εικόνα 4.3.1 : Οθόνη Σύνδεσης Χρήστη (LoginFragment)	36
Εικόνα 4.4.1 : Κεντρική Οθόνη Εγγράφων (DocumentsFragment).....	37
Εικόνα 4.4.2 : Tool Bar Menu (DocumentsFragment)	37
Εικόνα 4.4.3 : Delete Profile Confirmation Prompt (Documents Fragment).....	37
Εικόνα 4.5.1 : Προσθήκη ID Card.....	38
Εικόνα 4.5.2 : Προσθήκη Driver License	39
Εικόνα 4.5.3 : Προσθήκη Ticket	39
Εικόνα 4.5.4 : Επιλογή πηγής QR Code.....	39
Εικόνα 4.5.5 : Επιλογή πηγής φωτογραφίας ιδιόκτητη για ID Card και Driver's license.....	40
Εικόνα 4.5.6 : Επαλήθευση PIN.....	40
Εικόνα 4.5.7 : Επιτυχή εισαγωγή Document	41
Εικόνα 4.6.1 : ID CARD(DocumentDetailFragment).....	42
Εικόνα 4.6.2 : Driver's License(DocumentDetailFragment)	42
Εικόνα 4.6.3 : Ticket (DocumentDetailFragment)	43
Εικόνα 4.6.4 : Date picker (αλλαγή ήμερομηνίας λήξης).....	43
Εικόνα 4.6.5 : Ενημέρωση Ημερομηνίας Λήξης.....	44
Εικόνα 4.6.6 : Παρουσίαση QR Code (ShowQrDialogFragment)	44
Εικόνα 4.6.7 : Επαλήθευση PIN.....	45
Εικόνα 4.7.1 : Οθόνη Επαλήθευσης Χρήστη.....	45
Εικόνα 4.7.2 : Αποστολή Επαλήθευσης Email	46
Εικόνα 4.7.3 : Email Επαλήθευσης στο Gmail Inbox (ενδέχεται να σταλεί στο Spam)	46
Εικόνα 4.7.4 : Παράθυρο Επικόλλησης Συνδέσμου Επαλήθευσης	47
Εικόνα 4.7.5 : Αποστολή Mock Κωδικού Επαλήθευσης SMS	48
Εικόνα 4.7.6 : Παράθυρο Εισαγωγής Κωδικού Επαλήθευσης SMS	48
Εικόνα 4.7.7 : Οθόνη Εισαγωγής Παλαιού και Νέου PIN	49
Εικόνα 4.7.8 : Μήνυμα Επιτυχούς Αλλαγής PIN	50
Εικόνα 4.8.1 : Login fragment (Greek).....	51
Εικόνα 4.8.2 : Register fragment (German).....	51
Εικόνα 4.8.3 : Change pin (German).....	52
Εικόνα 4.8.4 : Add document fragment (Italian)	53
Εικόνα 4.8.5 : Add document fragment (French).....	54
Εικόνα 4.8.6 : Add document fragment (Spanish).....	55
Εικόνα 4.8.7 : Document details fragment (Greek).....	56
Εικόνα 4.8.8 : Pin prompt (Greek)	56
Εικόνα 4.8.9 : Delete account prompt (German).....	56

ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ

Πίνακας 3.2.1 : Λειτουργικές απαιτήσεις (Functional Requirements).....	23
Πίνακας 3.2.2 : Μη λειτουργικές απαιτήσεις (Non-Functional Requirements)	24
Πίνακας 3.7.1 : Επίπεδα αρχιτεκτονικής.....	28
Πίνακας 4.9.1 : Συσχέτιση με το Διάγραμμα Περιπτώσεων Χρήσης	56
Πίνακας 4.10.1 : Γενικά Μηνύματα	57
Πίνακας 4.10.2 : Οθόνη Σύνδεσης (LoginFragment)	57
Πίνακας 4.10.3 : Εγγραφή Χρήστη (RegisterFragment).....	58
Πίνακας 4.10.4 : Αλλαγή PIN (ChangePinFragment)	58
Πίνακας 4.10.5 : Προσθήκη Εγγράφου (AddDocumentFragment)	58
Πίνακας 4.10.6 : Λεπτομέρειες Εγγράφου (DocumentDetailFragment) ...	59
Πίνακας 4.10.7 : Κεντρική Οθόνη Εγγράφων (DocumentsFragment).....	59
Πίνακας 4.10.8 : Προβολή QR Εγγράφου (ShowQrDialogFragment).....	60
ΠΙΝΑΚΑΣ ΟΡΟΛΟΓΙΑΣ	66
ΣΥΝΤΜΗΣΕΙΣ – ΑΡΚΤΙΚΟΛΕΞΑ – ΑΚΡΩΝΥΜΙΑ	67

ΠΡΟΛΟΓΟΣ

Η παρούσα πτυχιακή εργασία εκπονήθηκε στο πλαίσιο των σπουδών μου στο Τμήμα Ψηφιακών Συστημάτων της Σχολής Τεχνολογιών Πληροφορικής και Επικοινωνιών του Πανεπιστημίου Πειραιώς.

Αντικείμενό της είναι η ανάπτυξη μιας εφαρμογής Ψηφιακού Πορτοφολιού για συσκευές Android, η οποία προσεγγίζει ως μελέτη περίπτωσης το Ευρωπαϊκό Ψηφιακό Πορτοφόλι Ταυτότητας (EUDI Wallet).

Η επιλογή του θέματος προέκυψε από το αυξανόμενο ενδιαφέρον για ζητήματα ασφαλείας, δια-λειτουργικότητας και προστασίας προσωπικών δεδομένων στην ψηφιακή εποχή, καθώς και από την επιθυμία μου να συνδυάσω τις τεχνολογικές μου γνώσεις με ένα έργο που διαθέτει πρακτική, κοινωνική και ευρωπαϊκή διάσταση.

Μέσα από την παρούσα εργασία επιχειρείται η διερεύνηση των τεχνολογικών και σχεδιαστικών προκλήσεων που συνοδεύουν την ανάπτυξη ενός ασφαλούς ψηφιακού πορτοφολιού, συμβάλλοντας έτσι στην καλύτερη κατανόηση των απαιτήσεων και δυνατοτήτων του συγκεκριμένου πεδίου.

Τέλος, το έργο αυτό μπορεί να θεωρηθεί ως ένα πρώτο βήμα προς τη μελλοντική υλοποίηση ενός πλήρως διαπιστευμένου ευρωπαϊκού προτύπου Ψηφιακού Πορτοφολιού Ταυτότητας.

1. ΕΙΣΑΓΩΓΗ

1.1 Πλαίσιο και Κίνητρο

Τα τελευταία χρόνια, η Ευρωπαϊκή Ένωση έχει προχωρήσει στην ανάπτυξη του Ευρωπαϊκού Πλαισίου Ψηφιακής Ταυτότητας (European Digital Identity Framework), το οποίο αποτελεί τη θεμελιώδη βάση για τη δημιουργία του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας (European Digital Identity Wallet).

Σκοπός του πλαισίου είναι να παρέχει σε κάθε πολίτη και κάτοικο της Ευρωπαϊκής Ένωσης έναν ασφαλή, διαλειτουργικό και πλήρως ελεγχόμενο από τον ίδιο τρόπο αποθήκευσης, διαχείρισης και παρουσίασης επαληθευμένων ψηφιακών διαπιστευτηρίων, όπως δελτία ταυτότητας, άδειες οδήγησης, φοιτητικές ή επαγγελματικές κάρτες, πιστοποιητικά, εισιτήρια και άλλα προσωπικά έγγραφα.

Η αυξανόμενη ανάγκη για ασφαλή ηλεκτρονική ταυτοποίηση και η επιτάχυνση της ψηφιακής διακυβέρνησης καθιστούν τέτοιου είδους λύσεις απαραίτητες σε εθνικό και ευρωπαϊκό επίπεδο. Οι πολίτες καλούνται καθημερινά να αποδεικνύουν την ταυτότητά τους σε πληθώρα δημόσιων και ιδιωτικών ψηφιακών υπηρεσιών, συχνά μέσω ετερογενών ή αποσπασματικών συστημάτων. Το ενιαίο Ευρωπαϊκό Ψηφιακό Πορτοφόλι, σύμφωνα με το πρότυπο eIDAS 2.0, στοχεύει στην ενοποίηση αυτών των μηχανισμών, διασφαλίζοντας διαφάνεια, φορητότητα, διαλειτουργικότητα και ενισχυμένο έλεγχο των προσωπικών δεδομένων από τον ίδιο τον χρήστη.

Η παρούσα εργασία επιχειρεί να επεκτείνει και να προσεγγίσει πρακτικά τις αρχές του παραπάνω πλαισίου μέσα από την υλοποίηση μιας απλοποιημένης αλλά λειτουργικά ρεαλιστικής εκδοχής του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητα, προσαρμοσμένης σε περιβάλλον Android.

Σε αντίθεση με τις υπάρχουσες θεωρητικές ή πειραματικές υλοποιήσεις, η προτεινόμενη εφαρμογή εστιάζει στην απλότητα, τη χρηστικότητα και την ασφάλεια, αξιοποιώντας σύγχρονες αρχιτεκτονικές προσεγγίσεις όπως το MVVM και τεχνολογίες μόνιμης αποθήκευσης δεδομένων όπως η Room Database.

Επιπλέον, η εργασία στοχεύει στη διερεύνηση βελτιωμένων πρακτικών ασφάλειας, μέσω κρυπτογράφησης PIN, τοπικής αποθήκευσης χωρίς εξάρτηση από εξωτερικούς διακομιστές, και πολυγλωσσικής υποστήριξης, ώστε να αντικατοπτρίζεται ο πανευρωπαϊκός και συμπεριληπτικός χαρακτήρας του συστήματος. Κατ' αυτόν τον τρόπο, η εφαρμογή δεν αποσκοπεί απλώς στην αναπαραγωγή του υπάρχοντος προτύπου, αλλά στη διερεύνηση πιο ευέλικτων, ασφαλών και φιλικών προς τον χρήστη λύσεων, που μπορούν να αποτελέσουν βάση για μελλοντική ενσωμάτωση ή εξέλιξη του ευρωπαϊκού ψηφιακού οικοσυστήματος ταυτοποίησης.

1.2 Αντικείμενο και Στόχοι της Εργασίας

Η παρούσα πτυχιακή εργασία έχει ως αντικείμενο τη μελέτη και υλοποίηση μιας απλοποιημένης αλλά λειτουργικά ολοκληρωμένης εκδοχής του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας (EUDI Wallet), προσαρμοσμένης για λειτουργία σε περιβάλλον Android. Η εργασία συνδυάζει θεωρητική διερεύνηση και πρακτική ανάπτυξη, αποτελώντας ταυτόχρονα μελέτη περίπτωσης (case study) και πιλοτική υλοποίηση μιας εφαρμογής που προσεγγίζει τις αρχές του ευρωπαϊκού πλαισίου ψηφιακής ταυτότητας.

Κεντρικός στόχος είναι η ανάπτυξη μίας ασφαλούς, αποδοτικής και φιλικής προς τον χρήστη εφαρμογής, η οποία θα επιτρέπει:

τη δημιουργία προσωπικού λογαριασμού και τη σύνδεση μέσω PIN,

- την τοπική αποθήκευση και διαχείριση προσωπικών εγγράφων,
- τη δυνατότητα προβολής, επεξεργασίας ή διαγραφής αυτών των εγγράφων,
- καθώς και την παρουσίαση του QR code για εύκολη και ασφαλή ταυτοποίηση.

Η εφαρμογή σχεδιάστηκε με βάση τη σύγχρονη αρχιτεκτονική MVVM (Model–View–ViewModel), ενώ η Room Database αξιοποιήθηκε για τη μόνιμη και αποδοτική αποθήκευση δεδομένων.

Ιδιαίτερη έμφαση δόθηκε σε ζητήματα ασφάλειας και προστασίας προσωπικών δεδομένων, μέσω της ενσωμάτωσης μηχανισμών ελέγχου ταυτότητας, κρυπτογράφησης PIN, καθώς και τοπικής αποθήκευσης χωρίς εξάρτηση από εξωτερικούς διακομιστές.

Παράλληλα, ενσωματώθηκε πολυγλωσσική υποστήριξη, ώστε η εφαρμογή να αντικατοπτρίζει τον πανευρωπαϊκό χαρακτήρα και τη διαλειτουργικότητα που διέπει το όραμα του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας.

Με τον τρόπο αυτό, η εργασία επιδιώκει όχι απλώς την αναπαραγωγή του υπάρχοντος προτύπου, αλλά τη διερεύνηση καινοτόμων και βελτιωμένων προσεγγίσεων, που θα μπορούσαν να συνεισφέρουν σε πιο ευέλικτες, ασφαλείς και σύγχρονες υλοποιήσεις του ευρωπαϊκού οικοσυστήματος ψηφιακής ταυτότητας.

1.3 Μεθοδολογία και Τεχνολογίες

Για την ανάπτυξη της εφαρμογής χρησιμοποιήθηκαν οι ακόλουθες τεχνολογίες:

- Γλώσσα προγραμματισμού: Kotlin
- Περιβάλλον ανάπτυξης: Android Studio
- Αρχιτεκτονική: Model-View-ViewModel (MVVM)
- Βάση δεδομένων: Room Database (SQLite)
- Διεπαφή χρήστη: Android XML layouts
- Μηχανισμοί ασφάλειας: Bcrypt για κατακερματισμό(hashing) PIN, επιβεβαίωση ταυτότητας, έλεγχος πρόσβασης

- Πολυγλωσσικότητα: Android localization system (strings.xml)

Η εφαρμογή δομήθηκε με σαφή διαχωρισμό επιπέδων (UI, ViewModel, Repository, DAO, Entity), ώστε να διασφαλίζεται η επεκτασιμότητα και η συντηρησιμότητά της.

1.4 Σχέση με την Αναφορά Υλοποίησης (Reference Implementation)

Η παρούσα υλοποίηση βασίστηκε στη μελέτη του “European Digital Identity Wallet Reference Implementation”, όπως αυτό είναι διαθέσιμο μέσω του επίσημου GitHub repository (αποθετηρίου) της Ευρωπαϊκής Επιτροπής [6]. Η συγκεκριμένη υλοποίηση αναφοράς αποτελεί σημείο αναφοράς για την κατανόηση της τεχνικής αρχιτεκτονικής και των προδιαγραφών του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας, λειτουργώντας ως κατευθυντήρια γραμμή για ερευνητές και προγραμματιστές που επιθυμούν να αναπτύξουν συμβατές ή επεκτάσιμες υλοποιήσεις.

Το Reference Implementation περιγράφει μία αρθρωτή (modular) αρχιτεκτονική, η οποία υποστηρίζει βασικές λειτουργίες, όπως:

- έκδοση και παρουσίαση διαπιστευτηρίων (credentials),
- έλεγχο εμπιστοσύνης (trust lists) και διαχείριση αξιόπιστων φορέων,
- καθώς και ασφαλή αποθήκευση και ανάκτηση κρυπτογραφικών κλειδίων.

Η προτεινόμενη εφαρμογή υιοθετεί τις ίδιες θεμελιώδεις αρχές σχεδίασης, όπως η ιδιωτικότητα, ασφάλεια, διαλειτουργικότητα και επεκτασιμότητα, και τις προσαρμόζει σε ένα απλοποιημένο, εκπαιδευτικό και πρακτικά εφαρμόσιμο πλαίσιο, το οποίο επικεντρώνεται στη λειτουργία του πορτοφολιού σε περιβάλλον Android.

Σε αντίθεση με την υλοποίηση αναφοράς, η παρούσα υλοποίηση δίνει έμφαση στη βελτιστοποίηση της εμπειρίας χρήστη, στην τοπική αποθήκευση δεδομένων χωρίς εξάρτηση από εξωτερικούς παρόχους, και στη διερεύνηση τρόπων ελαφρύτερης ενσωμάτωσης των μηχανισμών ασφαλείας. Με αυτόν τον τρόπο, λειτουργεί συμπληρωματικά προς την υλοποίηση αναφοράς, αναδεικνύοντας τις δυνατότητες προσαρμογής και ευελιξίας του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας σε διαφορετικά λειτουργικά περιβάλλοντα.

1.5 Δομή της Εργασίας

Η εργασία οργανώνεται στα ακόλουθα κεφάλαια:

Κεφάλαιο 2 – Θεωρητικό Υπόβαθρο: Παρουσιάζονται οι βασικές έννοιες και τεχνολογίες που αξιοποιήθηκαν, όπως το Android Jetpack, η αρχιτεκτονική MVVM, η Room Database, καθώς και οι αρχές του Repository pattern και της διαχείρισης δεδομένων μέσω DAO.

Κεφάλαιο 3 – Ανάλυση και Σχεδίαση του Συστήματος: Παρουσιάζεται η συνολική αρχιτεκτονική της εφαρμογής, τα διαγράμματα UML και το διάγραμμα οντοτήτων ER, καθώς και την συσχέτιση της υλοποίησης με τις κατευθυντήριες γραμμές του *European Digital Identity Wallet Reference Implementation*.

Κεφάλαιο 4 – Εγχειρίδιο Χρήσης: Παρουσιάζεται η λειτουργικότητα της εφαρμογής μέσω των Fragments της, όπως η εγγραφή (Register), η σύνδεση (Login), η διαχείριση εγγράφων (Documents, Add Document, Document Details) και η αλλαγή κωδικού PIN (Change PIN). Κάθε ενότητα συνοδεύεται από ενδεικτικά στιγμιότυπα οθόνης, καθώς και περιγραφή των ελέγχων εγκυρότητας και των προειδοποιητικών μηνυμάτων που εμφανίζονται στο περιβάλλον χρήστη.

Κεφάλαιο 5 – Συμπεράσματα και Μελλοντική Εργασία: Συνοψίζονται τα αποτελέσματα της υλοποίησης και εξάγονται τα βασικά συμπεράσματα που προέκυψαν από την ανάπτυξη της εφαρμογής. Παράλληλα, προτείνονται κατευθύνσεις για μελλοντική επέκταση και βελτίωση, σε συνάρτηση με τις επίσημες προδιαγραφές του Ευρωπαϊκού Ψηφιακού Πορτοφολιού.

Παράρτηματα: Περιλαμβάνουν επιλεγμένα αποσπάσματα κώδικα της εφαρμογής (Παράρτημα I), καθώς και το διάγραμμα πλοήγησης και ενδεικτικά παραδείγματα πολυγλωσσικής υποστήριξης (Παράρτημα II).

2. ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ

2.1 Ανάπτυξη Εφαρμογών Android με Kotlin

Η πλατφόρμα Android αποτελεί το πιο διαδεδομένο λειτουργικό σύστημα για κινητές συσκευές, με περισσότερους από τρία δισεκατομμύρια ενεργούς χρήστες παγκοσμίως. Η ανάπτυξη εφαρμογών Android γίνεται μέσα από το Android SDK (Software Development Kit) και υποστηρίζεται από ένα ολοκληρωμένο οικοσύστημα εργαλείων, βιβλιοθηκών και API που επιτρέπουν την πρόσβαση σε κάθε επίπεδο του συστήματος [3].

Η γλώσσα Kotlin, που εισήχθη επίσημα από τη Google το 2017, αποτελεί πλέον την προτεινόμενη γλώσσα ανάπτυξης για Android εφαρμογές [8].

Προσφέρει:

- Ασφάλεια τύπων (Type Safety) και διαχείριση null τιμών (Null Safety), αποτρέποντας κοινά σφάλματα της Java.
- Coroutines για αποδοτική ασύγχρονη εκτέλεση λειτουργιών, επιτρέποντας παράλληλη διαχείριση δεδομένων χωρίς μπλοκαρίσματα (non-blocking I/O).
- Συνοπτική σύνταξη (concise syntax), που βελτιώνει την αναγνωσιμότητα και μειώνει τον όγκο του κώδικα.
- Διαλειτουργικότητα (Interoperability) με τη Java, επιτρέποντας τη χρήση υπαρχόντων βιβλιοθηκών.

Το περιβάλλον Android Studio, που βασίζεται στο IntelliJ IDEA, παρέχει ισχυρά εργαλεία όπως Layout Editor, Gradle build system, Android Virtual Device (AVD) για δοκιμές, και υποστήριξη για Jetpack Components.

Στην παρούσα εργασία, η εφαρμογή Digital Wallet αναπτύχθηκε εξ ολοκλήρου με Kotlin και αξιοποίησε πλήρως το οικοσύστημα του Android Jetpack.

2.2 Το Οικοσύστημα του Android Jetpack

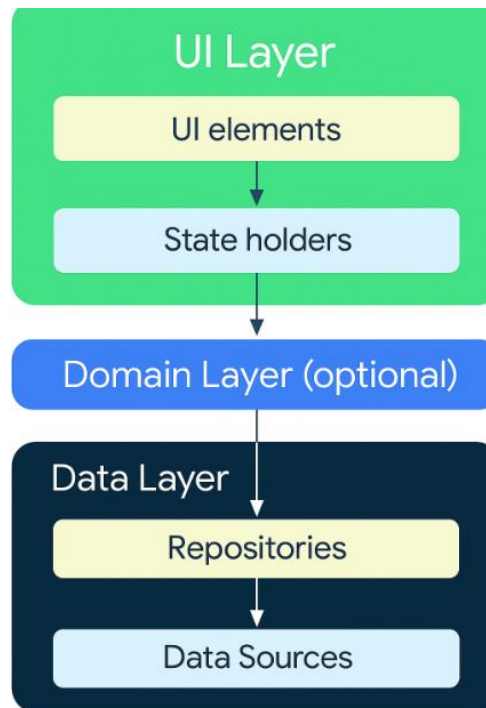
Το **Android Jetpack** είναι μια συλλογή από σύγχρονες βιβλιοθήκες της Google που βοηθούν στην ανάπτυξη εφαρμογών με δομημένο τρόπο, μειώνοντας το boilerplate code. Αποτελείται από τέσσερις βασικές κατηγορίες:

1. **Architecture** (Room, LiveData, ViewModel, DataStore)
2. **UI** (Fragments, Navigation, Compose)
3. **Behavior** (Permissions, Notifications, CameraX)
4. **Foundation** (AppCompat, Android KTX)

Η συγκεκριμένη εφαρμογή αξιοποιεί κυρίως τα **Architecture Components**, ώστε να ακολουθεί τη σύγχρονη αρχιτεκτονική MVVM. Με τον τρόπο αυτό επιτυγχάνεται **σαφής διαχωρισμός ρόλων, ευκολία συντήρησης, και ανθεκτικότητα** στις αλλαγές του κύκλου ζωής της εφαρμογής (Activity/Fragment lifecycle).

2.3 Η Αρχιτεκτονική MVVM (Model–View–ViewModel)

Η αρχιτεκτονική **MVVM (Model–View–ViewModel)** αποτελεί εξέλιξη του παραδοσιακού προτύπου **MVC (Model–View–Controller)**, προσαρμοσμένη για εφαρμογές με πλούσιο UI, όπως οι Android εφαρμογές.



Διάγραμμα 2.3.1: Αρχιτεκτονική MVVM [7]

Η λειτουργία της MVVM βασίζεται σε τρία διακριτά επίπεδα:

- **Model/Data Layer:** Περιλαμβάνει τις οντότητες δεδομένων, τη βάση δεδομένων και τα repositories που παρέχουν πρόσβαση στα δεδομένα. Είναι πλήρως ανεξάρτητο από τη διεπαφή χρήστη και μπορεί να επαναχρησιμοποιηθεί ή να ελεγχθεί αυτόνομα.
- **View/UI Elements:** Περιλαμβάνει το οπτικό μέρος της εφαρμογής: τα Fragments, Activities και τα layout αρχεία XML. Η View είναι υπεύθυνη μόνο για την εμφάνιση των δεδομένων και την αποστολή ενεργειών του χρήστη στο ViewModel.
- **ViewModel/State Holders:** Αποτελεί το ενδιάμεσο στρώμα που διατηρεί την κατάσταση της εφαρμογής και παρέχει δεδομένα στο View/UI μέσω **LiveData**. Χρησιμοποιεί coroutines για να εκτελεί ασύγχρονες λειτουργίες χωρίς να επηρεάζει το UI Thread.

Η χρήση του MVVM καθιστά την εφαρμογή:

- Ανθεκτική σε αλλαγές του UI lifecycle.

- Επεκτάσιμη, καθώς μπορούν να προστεθούν νέες λειτουργίες χωρίς να αλλάξει ο βασικός κορμός.
- Testable, καθώς κάθε στρώμα μπορεί να ελεγχθεί ανεξάρτητα.

2.4 Room Database και ORM προσέγγιση

Η Room αποτελεί μια βιβλιοθήκη Object–Relational Mapping (ORM) που ενσωματώνεται στο Android και έχει ως σκοπό να απλοποιήσει την επικοινωνία με τη βάση δεδομένων SQLite [10]. Παρέχει ένα επίπεδο αφάιρησης (abstraction layer) μεταξύ της εφαρμογής και της βάσης, επιτρέποντας την αναπαράσταση των πινάκων ως κλάσεις Kotlin, οι οποίες επισημαίνονται με annotations. Οι SQL εντολές υλοποιούνται μέσω Data Access Objects (DAO), εξασφαλίζοντας ασφαλή, δομημένη και ευκολότερα συντηρήσιμη πρόσβαση στα δεδομένα.

Ακολουθούν τα **κύρια πλεονεκτήματα της Room**:

- **Type safety**: Εντοπίζει πιθανά συντακτικά λάθη σε SQL queries κατά τη μεταγλώττιση (compile time), μειώνοντας τον κίνδυνο σφαλμάτων κατά την εκτέλεση.
- **Live queries**: Υποστηρίζει τη χρήση **Flow** ή **LiveData**, επιτρέποντας την **αυτόματη ενημέρωση του UI** κάθε φορά που αλλάζουν τα δεδομένα της βάσης.
- **Migrations**: Διευκολύνει την **αναβάθμιση της δομής της βάσης δεδομένων** χωρίς να υπάρχει απώλεια των ήδη αποθηκευμένων δεδομένων του χρήστη, μέσω ενός σαφούς μηχανισμού μεταβάσεων.
- **Σχέσεις (relations)**: Υποστηρίζει annotations όπως **@Relation**, **@Embedded** και **@ForeignKey**, που επιτρέπουν τη φυσική μοντελοποίηση σχέσεων μεταξύ οντοτήτων.

2.4.1. Ενδεικτική εφαρμογή στην εφαρμογή

Στην παρούσα εφαρμογή, η κλάση Document συσχετίζεται με την κλάση User μέσω ενός ξένου κλειδιού (userId), ενώ συνδέεται επίσης με επιμέρους οντότητες, όπως IDCard, Driver'sLicense και Ticket, με τη χρήση του τύπου εγγράφου (enum DocumentType). Χάρη στη Room, οι σχέσεις αυτές μπορούν να μοντελοποιηθούν απευθείας μέσα από τις κλάσεις Kotlin, χωρίς την ανάγκη χειροκίνητης διαχείρισης SQL εντολών ή πολύπλοκων συνδέσεων. Αυτό καθιστά την υλοποίηση πιο καθαρή, ασφαλή και επεκτάσιμη, βελτιώνοντας παράλληλα τη συντηρησιμότητα του κώδικα.

2.5 DAO (Data Access Object) Σχεδίαση

Κάθε οντότητα της βάσης δεδομένων συνοδεύεται από το δικό της DAO, το οποίο περιέχει όλες τις επιτρεπόμενες λειτουργίες (insert, update, delete, query). Τα DAOs της εφαρμογής είναι:

- **UserDao**: Διαχείριση χρηστών (εγγραφή, εύρεση βάσει email, αλλαγή PIN).
- **DocumentDao**: Εισαγωγή, ενημέρωση και ανάκτηση εγγράφων.
- **AuthLogDao**: Καταγραφή ενεργειών σύνδεσης/αποτυχίας.

Κάθε DAO περιλαμβάνει suspend functions, ώστε να εκτελούνται σε coroutine scope, χωρίς να μπλοκάρουν το κύριο νήμα.

2.6 Repository Pattern

Το Repository Pattern είναι κεντρικό στοιχείο του MVVM. Λειτουργεί ως ενιαίο σημείο πρόσβασης στα δεδομένα και απομονώνει τα ViewModels από τη βάση δεδομένων. Η λογική των repositories στην εφαρμογή παρουσιάζεται στη συνέχεια:

- **UserRepository:** Ελέγχει το email, τη διαθεσιμότητα PIN, επιβεβαιώνει χρήστες, και χρησιμοποιεί το Bcrypt για ασφαλή αποθήκευση.

Επικοινωνεί με το AuthLogDao για να καταγράφει επιτυχημένες ή αποτυχημένες προσπάθειες σύνδεσης.

- **DocumentRepository:** Συνδυάζει ελέγχους εγκυρότητας (QR, ημερομηνίες, τύπους εγγράφων) και εκτελεί συναλλαγές (withTransaction) ώστε να διασφαλίζεται η ατομικότητα των ενεργειών. Με αυτόν τον τρόπο, το Repository Layer συγκεντρώνει όλη τη “λογική της επιχείρησης” (business logic), ενώ το ViewModel περιορίζεται στη διαχείριση της διεπαφής.

2.7 ViewModel και LiveData

Τα **ViewModels** διαχειρίζονται την κατάσταση του UI και παρέχουν δεδομένα με ασύγχρονο τρόπο.

Η εφαρμογή περιλαμβάνει δύο κύρια ViewModels:

- **AuthViewModel:** για διαδικασίες εγγραφής, σύνδεσης, αλλαγής PIN και διαγραφής λογαριασμού.
- **DocumentViewModel:** για τη διαχείριση των εγγράφων, προβολή, προσθήκη και ενημέρωση ημερομηνιών λήξης.

Τα ViewModels αξιοποιούν:

- **LiveData:** παρατηρήσιμα δεδομένα που ενημερώνουν το UI όταν αλλάζει κάτι στη βάση.
- **viewModelScope:** ασφαλής εκτέλεση coroutines στο πλαίσιο του κύκλου ζωής του ViewModel.

Η επιλογή αυτή διασφαλίζει **αποσύνδεση λογικής και παρουσίασης**, επιτρέποντας στην εφαρμογή να παραμένει σταθερή ακόμη και μετά από αλλαγές κατάστασης (π.χ. rotation, backgrounding).

2.8 Ασφάλεια και Διαχείριση Συνεδριών

Η ασφάλεια δεδομένων είναι κρίσιμη σε ένα ψηφιακό πορτοφόλι. Η εφαρμογή εφαρμόζει τις παρακάτω αρχές [12]:

1. **Hashing PINs:** Το PIN του χρήστη δεν αποθηκεύεται ποτέ ως απλό κείμενο. Χρησιμοποιείται **Bcrypt** για ισχυρό κατακερματισμό.
2. **AuthLog:** Κάθε ενέργεια σύνδεσης ή αποτυχίας καταγράφεται με *timestamp*, επιτρέποντας την παρακολούθηση ύποπτης δραστηριότητας.
3. **Lockout mechanism:** Μετά από τρεις αποτυχημένες προσπάθειες, ο λογαριασμός κλειδώνει προσωρινά (15 λεπτά).
4. **SessionManager:** Διαχειρίζεται την τρέχουσα συνεδρία με *SharedPreferences*, επιτρέποντας προσωρινή αποθήκευση *userId* χωρίς αποθήκευση ευαίσθητων στοιχείων.
5. **Firebase Authentication:** Για τη διαδικασία ταυτοποίησης χρηστών αξιοποιείται το σύστημα **Firebase Authentication** της πλατφόρμας Google, το οποίο υποστηρίζει επαλήθευση μέσω **email** και **τηλεφώνου**. Η αποστολή email επιβεβαίωσης λειτουργεί κανονικά, ενώ η επαλήθευση SMS προσομοιώνεται μέσω **mock μηχανισμού** λόγω περιορισμών του δωρεάν επιπέδου πρόσβασης στο Firebase [9].
Σε μελλοντική έκδοση προβλέπεται η ενσωμάτωση **πλήρους μηχανισμού επαλήθευσης SMS**, με αποθήκευση και έλεγχο του τηλεφωνικού αριθμού στη βάση δεδομένων, όπως αναφέρεται στο Κεφάλαιο 5.
6. **Privacy by Design:** Όλα τα δεδομένα παραμένουν τοπικά στη συσκευή. Δεν γίνεται μεταφορά ή συγχρονισμός με εξωτερικούς *servers*.

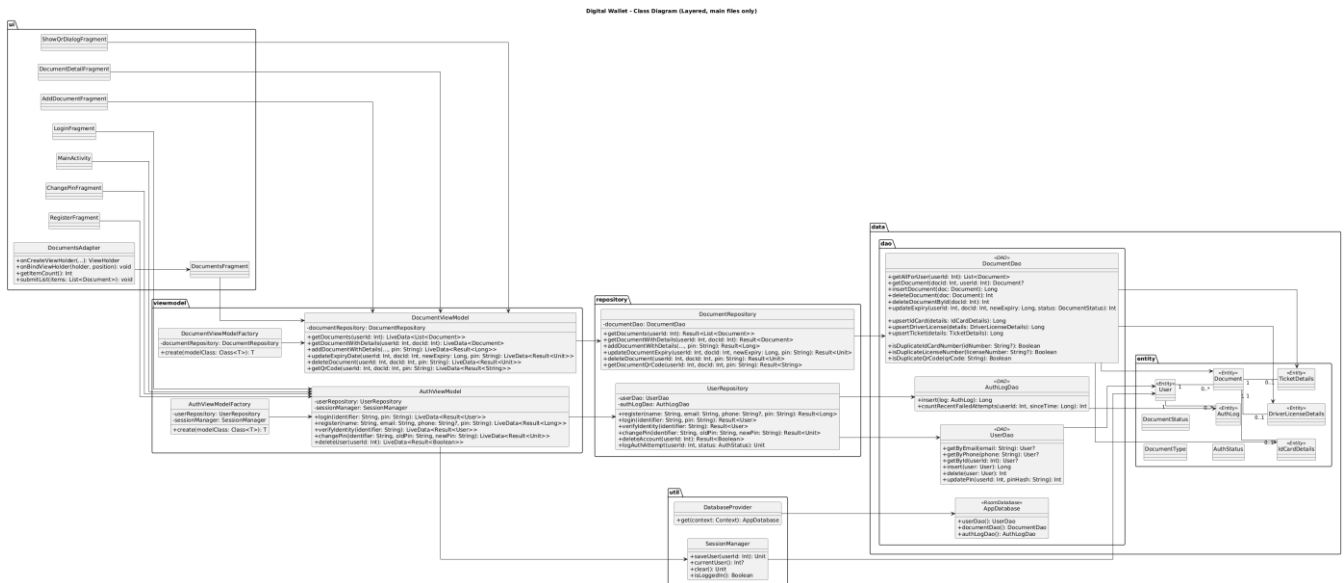
Τεχνική Σημείωση: Το Firebase Authentication χρησιμοποιείται αποκλειστικά για την επαλήθευση email κατά τη διαδικασία εγγραφής και αλλαγής PIN. Δεν αποθηκεύει δεδομένα της εφαρμογής και δεν συμμετέχει στη ροή δεδομένων MVVM. Για τον λόγο αυτό καλείται απευθείας από το UI Layer, ενώ όλη η επιχειρησιακή λογική και το authentication μέσω PIN παραμένουν στο Repository και στην Room Database.

2.9 Συνολική Αρχιτεκτονική και Ροή Δεδομένων

Η ροή δεδομένων από τον χρήστη προς τη βάση ακολουθεί τα εξής βήματα:

1. Ο χρήστης εκτελεί μια ενέργεια στο UI (π.χ. “Προσθήκη Εγγράφου”).
2. Το **Fragment** ειδοποιεί το **ViewModel**.
3. Το **ViewModel** καλεί τη μέθοδο του **Repository**, το οποίο επικοινωνεί με το **DAO**.
4. Το **DAO** αλληλεπιδρά με τη **Room Database** και επιστρέφει το αποτέλεσμα.
5. Το **ViewModel** ενημερώνει το **LiveData**, το οποίο ενημερώνει αυτόματα το UI.

Η σχεδίαση αυτή εξασφαλίζει πλήρη **ενθυλάκωση (encapsulation)** και **ανεξαρτησία επιπέδων**.



Διάγραμμα 2.9.1: Αρχιτεκτονική και Ροή Δεδομένων

Το διάγραμμα αυτό απεικονίζει τη ροή επικοινωνίας μεταξύ των επιπέδων UI, ViewModel, Repository, DAO και Database.

2.10 Συνοπτική Αξιολόγηση της Αρχιτεκτονικής

Συνοψίζοντας την ανάλυση των προηγούμενων ενοτήτων, η εφαρμογή ακολουθεί πλήρως τις ακόλουθες αρχές [1] [5]:

- **Διαχωρισμός ευθυνών (Separation of Concerns):** Απεξάρτηση του UI Layer από την επιχειρησιακή λογική του Data Layer.
- **Αρχή Ενιαίας Ευθύνης (Single Responsibility Principle):** Ένας ξεχωριστός ρόλος για κάθε κλάση.
- **Επαναχρησιμοποίηση κώδικα (Code Reusability):** Χρήση κοινών Components για μείωση του κώδικα.
- **Δοκιμασιμότητα (Testability):** Ανεξάρτητος έλεγχος κάθε επιπέδου της εφαρμογής.
- **Ασφάλεια και επεκτασιμότητα (Security & Extensibility):** Εύκολη και ασφαλής ενσωμάτωση νέων λειτουργιών χωρίς αλλαγές στον πυρήνα της εφαρμογής.

Παράλληλα, η εφαρμογή πληροί βασικά στοιχεία του **EUDI Reference Implementation**:

- Modular design: Δομοστοιχείωση με σκοπό την ευκολότερη συντήρηση της εφαρμογής
- Privacy by design: Προστασία των δεδομένων του χρήστη εκ σχεδιασμού
- Local-first data handling: Αποκλειστικά τοπική επεξεργασία των δεδομένων.
- Multilingual and user-centric UX: Φιλική προς τον χρήστη διεπαφή με πλήρη υποστήριξη γλωσσών.

3. ΑΝΑΛΥΣΗ ΚΑΙ ΣΧΕΔΙΑΣΗ ΤΟΥ ΣΥΣΤΗΜΑΤΟΣ

3.1 Εισαγωγή

Το κεφάλαιο αυτό αναλύει τη σχεδίαση και την εσωτερική αρχιτεκτονική της εφαρμογής **Digital Wallet**, η οποία υλοποιήθηκε ως μελέτη περίπτωσης του **Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας (EUDI Wallet)**. Η ανάλυση περιλαμβάνει τα **UML διαγράμματα**, τη **δομή των δεδομένων**, τις **λειτουργικές ροές**, και τη **συσχέτιση** των επιμέρους τμημάτων του λογισμικού (UI, ViewModel, Repository, DAO, Database).

Επιπλέον, το κεφάλαιο εξετάζει τη **λογική αλληλεπίδρασης** μεταξύ των ενοτήτων του συστήματος, τους **μηχανισμούς ασφάλειας** και τον τρόπο με τον οποίο η εφαρμογή ευθυγραμμίζεται με τις αρχές του **Reference Implementation** της Ευρωπαϊκής Επιτροπής [6].

3.2 Ανάλυση Απαιτήσεων Συστήματος

Η ανάλυση των απαιτήσεων αποτελεί το πρώτο στάδιο της σχεδίασης. Από τη βιβλιογραφική μελέτη του ευρωπαϊκού πλαισίου **EUDI Wallet**, προέκυψε η ανάγκη δημιουργίας ενός ψηφιακού πορτοφολιού που θα προσφέρει ασφαλή, αποκεντρωμένη και ελεγχόμενη πρόσβαση στα προσωπικά διαπιστευτήρια του χρήστη.

Οι απαιτήσεις χωρίστηκαν σε **λειτουργικές** και **μη λειτουργικές** [4] και εμφανίζονται στους παρακάτω πίνακες:

Πίνακας 3.2.1 : Λειτουργικές απαιτήσεις (Functional Requirements)

Κωδικός	Περιγραφή Λειτουργίας	Επεξήγηση
F1	Εγγραφή νέου χρήστη	Ο χρήστης εισάγει email και PIN, τα οποία αποθηκεύονται με κατακερματισμό (Bcrypt).
F2	Σύνδεση χρήστη	Το σύστημα επαληθεύει τα διαπιστευτήρια και επιτρέπει πρόσβαση στο wallet.
F3	Επαλήθευση email/τηλεφώνου	Χρησιμοποιείται πριν από κρίσιμες ενέργειες (όπως αλλαγή PIN).
F4	Προσθήκη εγγράφου	Ο χρήστης καταχωρεί νέα έγγραφα (ID, δίπλωμα, εισιτήριο) με έλεγχο εγκυρότητας.
F5	Προβολή/παρουσίαση εγγράφου	Παρουσίαση μέσω QR code ή πλήρους προεπισκόπησης.
F6	Ενημέρωση ημερομηνίας λήξης	Επεξεργασία πεδίου ημερομηνίας με επαλήθευση κανόνων.
F7	Διαγραφή εγγράφου	Διαγραφή εγγράφου μετά από επιβεβαίωση.

Κωδικός	Περιγραφή Λειτουργίας	Επεξήγηση
F8	Διαγραφή λογαριασμού	Οριστική διαγραφή όλων των δεδομένων του χρήστη.
F9	Πολυγλωσσικότητα	Η εφαρμογή πρέπει να εμφανίζει περιεχόμενο σε 6 ευρωπαϊκές γλώσσες.

Πίνακας 3.2.2: Μη λειτουργικές απαιτήσεις (Non-Functional Requirements)

Κατηγορία	Περιγραφή	Επεξήγηση
Ασφάλεια	Προστασία δεδομένων	Όλα τα PINs κατακερματίζονται με Bcrypt. Καμία πληροφορία δεν μεταδίδεται εκτός συσκευής.
Απόδοση	Χαμηλή καθυστέρηση	Οι βάσεις δεδομένων λειτουργούν ασύγχρονα μέσω coroutines.
Επεκτασιμότητα	Modular σχεδίαση	Δυνατότητα προσθήκης νέων τύπων εγγράφων χωρίς τροποποίηση της βάσης.
Αξιοπιστία	Ανθεκτικότητα δεδομένων	Κάθε τροποποίηση εγγράφου εκτελείται μέσα σε συναλλαγή (transaction).
Χρηστικότητα	Φιλικό περιβάλλον	Καθαρή πλοήγηση, γρήγορη πρόσβαση στις βασικές λειτουργίες, απλή ορολογία.

Οι παραπάνω απαιτήσεις καθοδήγησαν τη λήψη όλων των σχεδιαστικών αποφάσεων και αποτέλεσαν τη βάση των UML διαγραμμάτων που ακολουθούν.

3.3 Περιγραφή Αρχιτεκτονικής

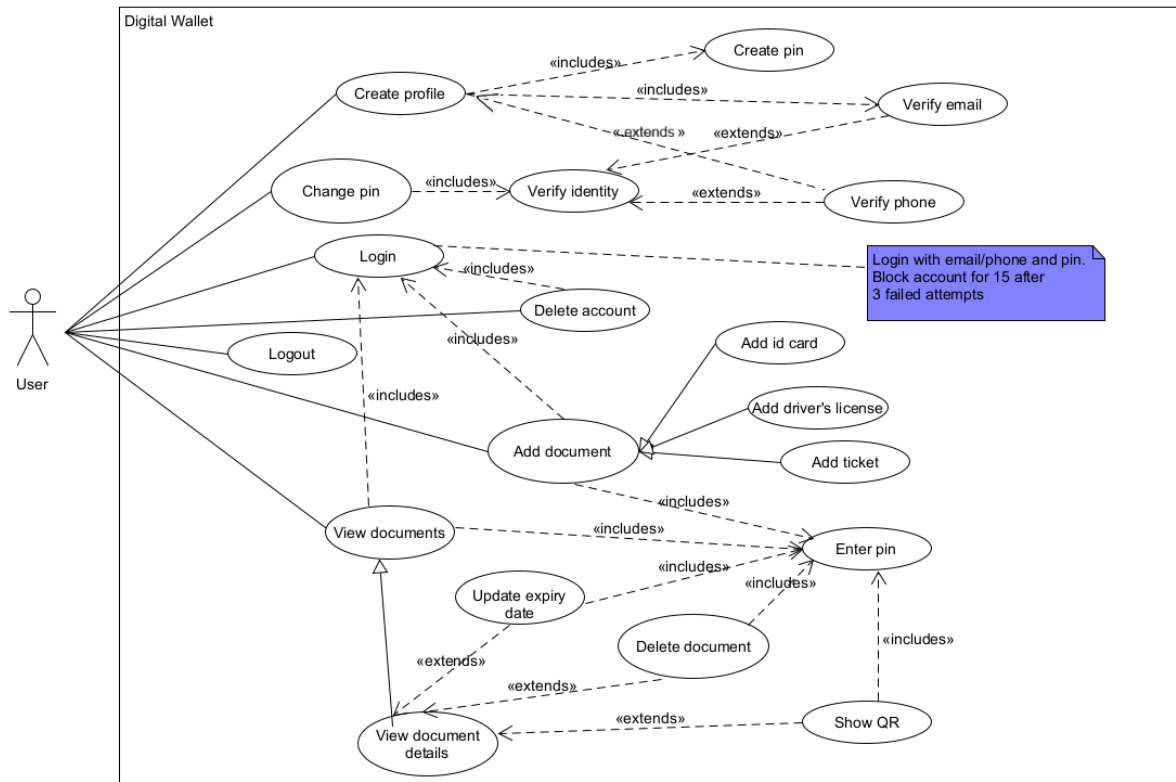
Η εφαρμογή Digital Wallet βασίζεται σε πολυεπίπεδη αρχιτεκτονική (multi-layered architecture), η οποία διασφαλίζει σαφή διαχωρισμό ευθυνών και ανεξαρτησία των επιπέδων. Τα βασικά στρώματα της αρχιτεκτονικής είναι τα ακόλουθα:

1. **User Interface (UI Layer):** Περιλαμβάνει τα Fragments και Activities (Login, Register, Add Document, κ.λπ.) και διαχειρίζεται την αλληλεπίδραση του χρήστη με το σύστημα.
2. **ViewModel Layer:** Παρέχει δεδομένα στο UI, διατηρεί το state της εφαρμογής, και εκτελεί ασύγχρονες λειτουργίες. Επικοινωνεί με τα Repositories και ενημερώνει το UI μέσω LiveData.
3. **Repository Layer:** Αποτελεί το σημείο σύνδεσης μεταξύ ViewModel και DAO. Περιέχει όλη τη “λογική της επιχείρησης” (business logic) όπως επαλήθευση PIN, χρονικοί έλεγχοι, και επικύρωση δεδομένων.
4. **Data Layer (DAO):** Περιέχει τις μεθόδους πρόσβασης στη βάση δεδομένων (insert, update, delete, query).
5. **Database Layer (Room Entities):** Περιλαμβάνει τις οντότητες (User, Document, AuthLog, κ.λπ.) και τις μεταξύ τους σχέσεις.

Η επικοινωνία είναι **κατακόρυφη και μονόδρομη**:

UI → ViewModel → Repository → DAO → Database
και πίσω μέσω **LiveData** για ενημέρωση του UI.

3.4 Διάγραμμα Περιπτώσεων Χρήσης (Use Case Diagram)



Διάγραμμα 3.4.1: UML Use Case Diagram [2]

Το Use Case Diagram αποτυπώνει τις αλληλεπιδράσεις μεταξύ του χρήστη και των κύριων λειτουργιών.

Ο χρήστης (Actor: User) μπορεί να εκτελέσει ενέργειες όπως:

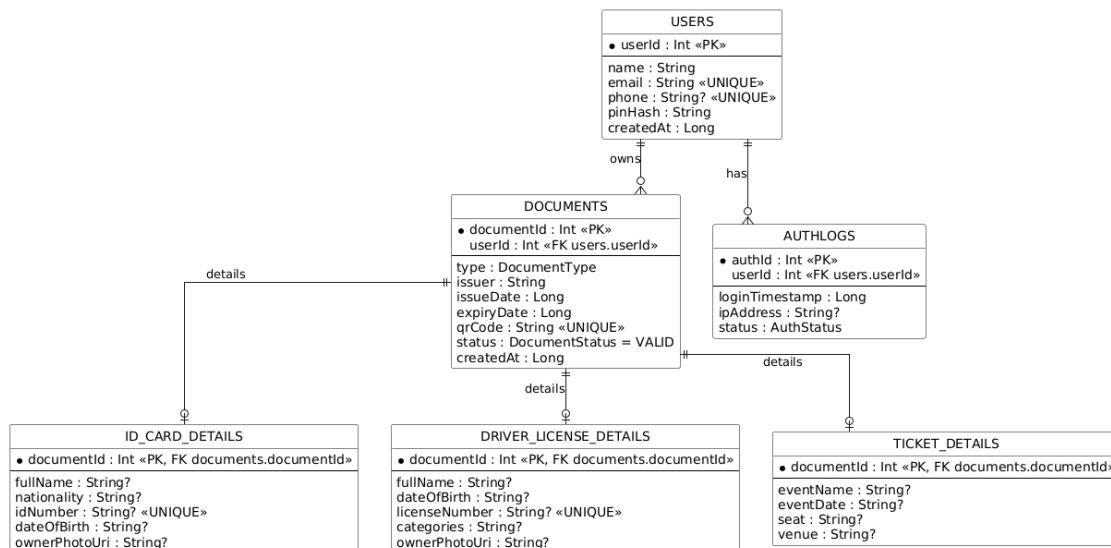
- Εγγραφή/Σύνδεση/Διαγραφή προφίλ/Αποσύνδεση
- Διαχείριση εγγράφων (Προσθήκη, Προβολή, Ενημέρωση, Διαγραφή)
- Αλλαγή PIN
- Παρουσίαση μοναδικού QR code ενός εγγράφου.

Οι σχέσεις <<include>> και <<extend>> χρησιμοποιούνται για να δηλώσουν ότι:

- Η “προβολή λεπτομερειών ενός εγγράφου” (**View Document Details**) είναι εξειδίκευση της “προβολή των εγγράφων του χρήστη” (**View Documents**).
- Η προσθήκη εγγράφου μπορεί να γενικεύεται από 3 διαφορετικούς τύπους εγγράφων (ID card, driver’s license, ticket).
- Η αλλαγή PIN μπορεί να γίνει είτε με επαλήθευση email είτε επαλήθευση τηλεφώνου (mutually exclusive extensions).

Η σχεδίαση αντικατοπτρίζει την αρχή του **ελάχιστου δικαιώματος (Least Privilege)**, καθώς κάθε ευαίσθητη ενέργεια, όπως η προσθήκη εγγράφου και η παρουσίαση QR, απαιτεί προηγούμενη επαλήθευση μέσω PIN (includes Enter PIN).

3.5 Δομή και Σχέσεις Δεδομένων – ER Διάγραμμα



Διάγραμμα 3.5.1: ER Diagram

Το ER διάγραμμα απεικονίζει τις σχέσεις μεταξύ των οντοτήτων της βάσης:

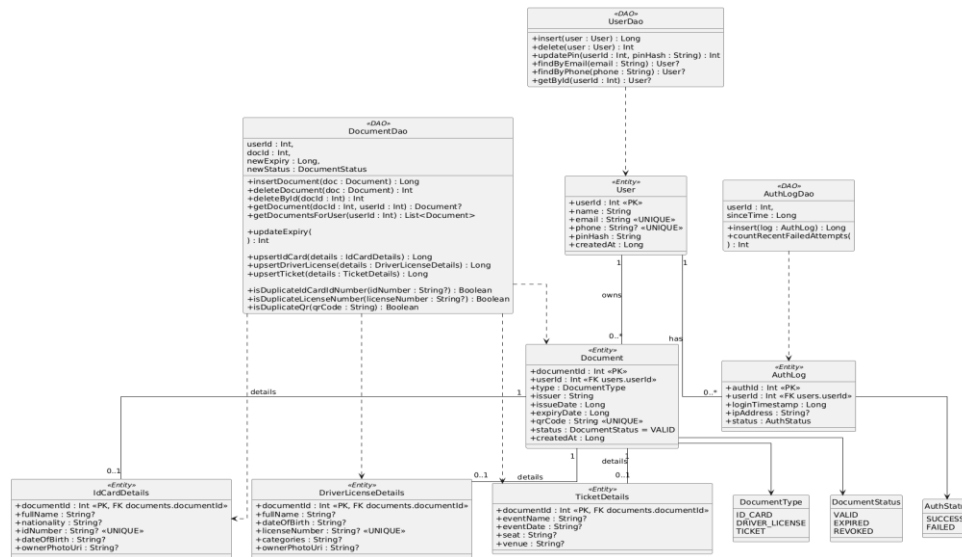
- **User (1) ↔ Document (N)**
- **Document (1) ↔ DocumentDetails (1)**
- **User (1) ↔ AuthLog (N)**

Η σχεδίαση ακολουθεί το πρότυπο **1:N** μεταξύ χρηστών και εγγράφων, καθώς κάθε χρήστης μπορεί να κατέχει πολλά έγγραφα.

Κάθε έγγραφο συνδέεται με έναν πίνακα λεπτομερειών, ανάλογα με τον τύπο του. Συγκεκριμένα, υπάρχουν τρεις πίνακες λεπτομερειών (IdCardDetails, DriverLicenseDetails, TicketDetails), οι οποίοι αποθηκεύουν τα επιπλέον χαρακτηριστικά του εκάστοτε τύπου εγγράφου (π.χ. idNumber, licenseNumber, eventDate, seat).

Η οντότητα **AuthLog** παρέχει μηχανισμό auditing, καταγράφοντας κάθε απόπειρα σύνδεσης με χρονική σήμανση.

3.6 UML Διάγραμμα Κλάσεων Βάσης Δεδομένων



Διάγραμμα 3.6.1: UML Database Class Diagram [2]

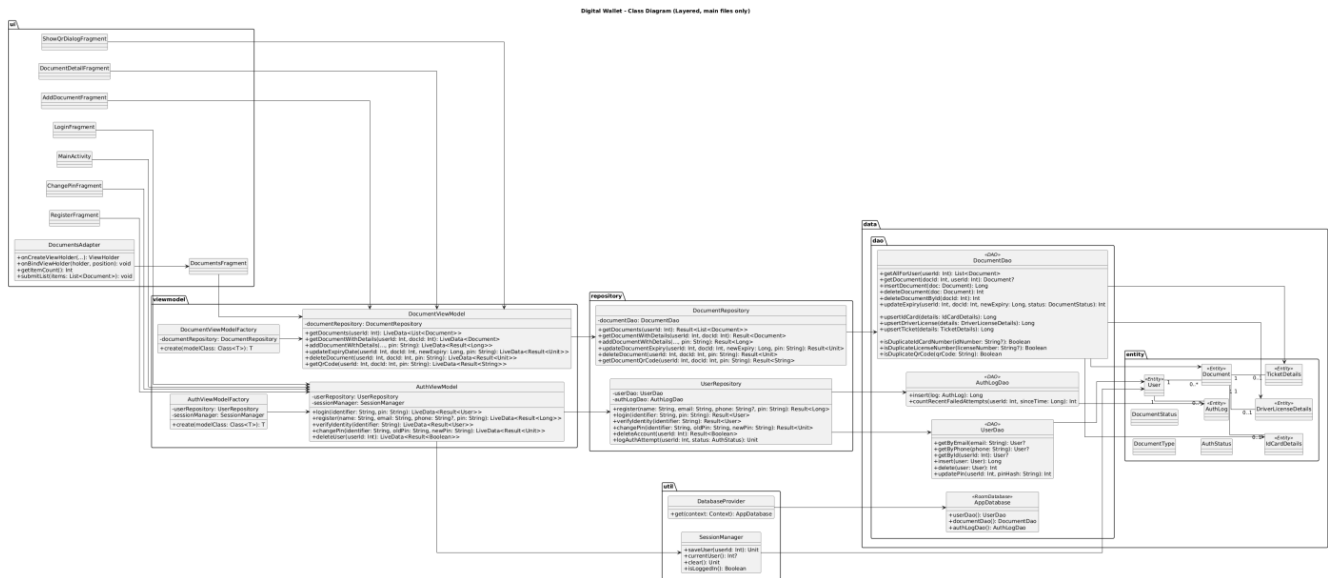
Το UML διάγραμμα δείχνει την αντικειμενοστραφή δομή της βάσης και τις σχέσεις εξάρτησης μεταξύ των κλάσεων User, Document, DocumentDetails, AuthLog και των DAO τους.

Κάθε οντότητα περιλαμβάνει:

- @PrimaryKey (μοναδικό αναγνωριστικό)
- @ForeignKey (σχέσεις)
- Περιορισμούς μοναδικότητας (unique)
- Enum τύπους για τις καταστάσεις των εγγράφων (DocumentStatus)

Η σχεδίαση εξασφαλίζει **συνέπεια (consistency)**, **ακεραιότητα (integrity)** και **ανθεκτικότητα (durability)**.

3.7 Επίπεδα Αρχιτεκτονικής – Layered Diagram



Διάγραμμα 3.7.1: UML Layered Class Diagram [2]

Το διάγραμμα αυτό παρουσιάζει τα επίπεδα της αρχιτεκτονικής, τα οποία αναλύονται στον επόμενο πίνακα:

Πίνακας 3.7.1 : Επίπεδα αρχιτεκτονικής

Επίπεδο	Ρόλος	Αντιπροσωπευτικές Κλάσεις
UI Layer	Αλληλεπίδραση χρήστη	Fragments (Login, Documents, AddDocument etc.)
ViewModel Layer	Διαχείριση κατάστασης	AuthViewModel, DocumentViewModel, AuthViewModelFactory, DocumentViewModelFactory
Repository Layer	Επιχειρησιακή λογική	UserRepository, DocumentRepository
DAO Layer	Πρόσβαση δεδομένων	UserDao, DocumentDao, AuthLogDao
Database Layer	Αποθήκευση & Σχέσεις	AppDatabase, Entities

Η ροή των δεδομένων είναι **bottom-up** και **reactive**, μέσω LiveData. Η μοντελοποίηση αυτή επιτρέπει μελλοντικά την αντικατάσταση της τοπικής βάσης με **cloud storage** ή **remote trust services**, χωρίς αλλαγή στην υπόλοιπη αρχιτεκτονική.

3.8 Ανάλυση Κύριων Συνιστωσών

Ακολουθούν οι κύριες συνιστώσες

Fragments & Main Activity:

- Παρέχουν το UI και καλούν το ViewModel για ενέργειες.
- Χρησιμοποιούν **Navigation Component** για μεταβάσεις μεταξύ οθονών.
- Διαχειρίζονται δυναμικά το state (π.χ. εισαγωγή φωτογραφίας ή QR).

ViewModels:

- Περιλαμβάνουν LiveData για ενημέρωση της διεπαφής χρήστη.
- Επικοινωνούν με τα Repositories μέσω coroutines.
- Παρέχουν προστασία από memory leaks και rotation events.

Repositories:

- Περιέχουν κανόνες επικύρωσης (validation rules) και ασφάλειας.
- Συνδυάζουν πολλαπλά DAOs για σύνθετες συναλλαγές.
- Αποτελούν σημείο ενσωμάτωσης μελλοντικών υπηρεσιών (π.χ. API).

DAOs & Database:

- Εκτελούν όλες τις CRUD ενέργειες με ασφάλεια τύπων.
- Υποστηρίζουν migrations και cascade delete για διασφάλιση συνέπειας.

Utilities (SessionManager, DatabaseProvider, Adapter):

- Παρέχουν επαναχρησιμοποιήσιμες λειτουργίες, όπως διαχείριση συνεδρίας και προσαρμογή λιστών.

3.9 Πολυγλωσσική Υλοποίηση

Η εφαρμογή αναπτύχθηκε με πλήρη τοπικοποίηση (localization) σε έξι γλώσσες: Αγγλικά (default γλώσσα), Ελληνικά, Γερμανικά, Ιταλικά, Ισπανικά και Γαλλικά.

Η πολυγλωσσικότητα υλοποιήθηκε μέσω του αρχείου strings.xml του Android, το οποίο περιλαμβάνει όλες τις μεταφράσεις των μηνυμάτων, τίτλων, ειδοποιήσεων και στοιχείων διεπαφής της εφαρμογής. Με αυτόν τον τρόπο επιτυγχάνεται προσαρμογή του περιβάλλοντος χρήσης ανάλογα με τη γλώσσα του συστήματος ή την προτίμηση του χρήστη, ενισχύοντας τη χρηστικότητα και την προσβασιμότητα σε διαφορετικά πολιτισμικά και γλωσσικά πλαίσια.

Επιπλέον, στο ίδιο αρχείο ενσωματώθηκε πίνακας κρατών-μελών της Ευρωπαϊκής Ένωσης, μέσω του οποίου ο χρήστης μπορεί να επιλέγει τη χώρα έκδοσης των προσωπικών του εγγράφων. Η λειτουργία αυτή προσδίδει στην εφαρμογή ρεαλιστικό και επεκτάσιμο χαρακτήρα, προσομοιώνοντας τη διακρατική διάσταση και πανευρωπαϊκή λειτουργικότητα που προδιαγράφεται

στο πλαίσιο του Ευρωπαϊκού Ψηφιακού Πορτοφολιού Ταυτότητας (EUDI Wallet).

Η πολυγλωσσική υποστήριξη δεν αποτελεί μόνο αισθητική ή λειτουργική επιλογή, αλλά σημαντική προϋπόθεση διαλειτουργικότητας και ενσωμάτωσης σε ένα ευρωπαϊκό οικοσύστημα ταυτοποίησης που στηρίζεται στην πολιτισμική και γλωσσική ποικιλομορφία των κρατών-μελών

3.10 Ευθυγράμμιση με το EUDI Reference Implementation

Η σχεδίαση ακολουθεί τις ακόλουθες βασικές αρχές του **EUDI Wallet Architecture & Reference Framework**:

- **User-Controlled Data:** Ο χρήστης είναι ο αποκλειστικός διαχειριστής των δεδομένων του.
- **Local-First Design:** Όλα τα δεδομένα φυλάσσονται στη συσκευή.
- **Security-by-Design:** PIN hashing, auditing, και ελάχιστα δικαιώματα πρόσβασης.
- **Interoperability:** Modular δομή και δυνατότητα προσαρμογής σε μελλοντικά ευρωπαϊκά APIs (OpenID4VCI, OpenID4VP).
- **Multilingual UX:** Συμμόρφωση με την πολυγλωσσική ευρωπαϊκή πολιτική (EN, EL, DE, FR, IT, ES).

4. ΕΓΧΕΙΡΙΔΙΟ ΧΡΗΣΗΣ ΤΗΣ ΕΦΑΡΜΟΓΗΣ

4.1 Εισαγωγή

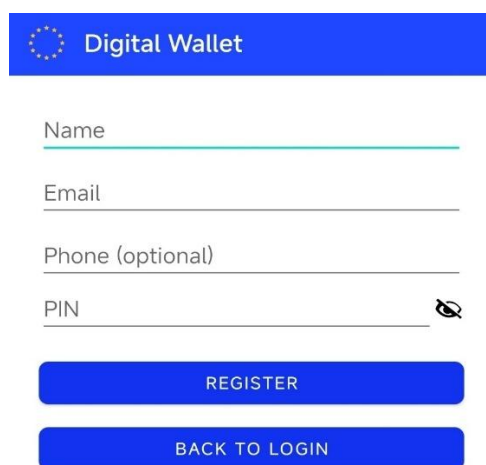
Το κεφάλαιο αυτό παρουσιάζει τη χρήση της εφαρμογής **Digital Wallet** από την οπτική του τελικού χρήστη και περιγράφει λεπτομερώς τη λειτουργία κάθε διεπαφής.

Η ανάλυση ακολουθεί τις περιπτώσεις χρήσης του **Διαγράμματος 3.4.1 : Διάγραμμα Περιπτώσεων Χρήσης**, εξηγώντας τη ροή ενεργειών του χρήστη, τους κανόνες ασφάλειας, καθώς και τα μηνύματα ενημέρωσης και σφαλμάτων που εμφανίζονται στο περιβάλλον εργασίας.

4.2 Οθόνη Εγγραφής (RegisterFragment)

Περίπτωση Χρήσης: Create Profile / Verify Identity / Verify Email / Verify Phone

Η οθόνη εγγραφής επιτρέπει τη δημιουργία νέου προφίλ χρήστη στην εφαρμογή. Ο χρήστης εισάγει το ονοματεπώνυμο, τη διεύθυνση email, προαιρετικά τον αριθμό τηλεφώνου και έναν προσωπικό κωδικό PIN. Κατά την ολοκλήρωση, ενεργοποιούνται οι διαδικασίες επαλήθευσης ταυτότητας μέσω email και SMS, σύμφωνα με τη ροή του διαγράμματος περιπτώσεων χρήσης.



Digital Wallet

Name

Email

Phone (optional)

PIN

REGISTER

BACK TO LOGIN

Εικόνα 4.2.1 : Οθόνη Εγγραφής (RegisterFragment)

Με την επιλογή του κουμπιού **REGISTER**, η εφαρμογή ελέγχει την εγκυρότητα των πεδίων (μη κενά πεδία, έγκυρη μορφή email, μήκος PIN μεταξύ 4 και 8 ψηφίων). Αν τα δεδομένα είναι ορθά, τα input fields απενεργοποιούνται, το κουμπί register αλλάζει σε continue, αποστέλλεται email επαλήθευσης στον χρήστη και, εφόσον έχει δηλωθεί αριθμός τηλεφώνου, αποστέλλεται επίσης SMS με κωδικό επιβεβαίωσης μετά την επαλήθευση του email.

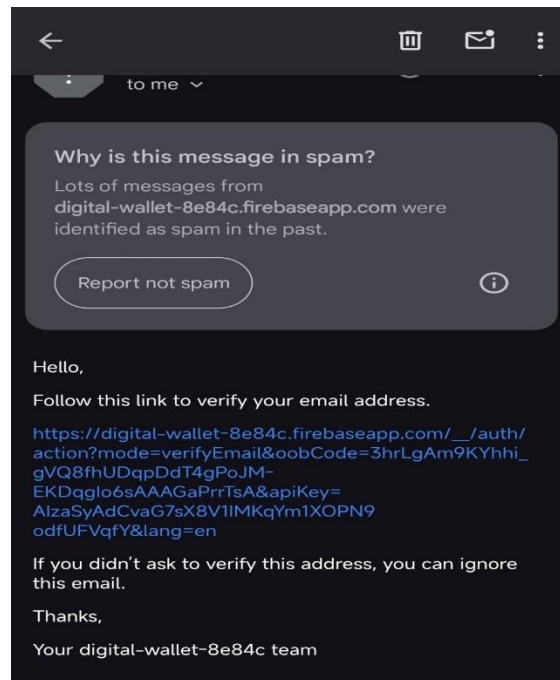
Εικόνα 4.2.2 : Αποστολή Επαλήθευσης Email

Επαλήθευση μέσω Email

Παράλληλα, αποστέλλεται email επαλήθευσης στον λογαριασμό του χρήστη. Ο χρήστης ακολουθεί τον σύνδεσμο που παρέχεται στο μήνυμα για να ολοκληρωθεί η επιβεβαίωση. Μετά την επιτυχή ενεργοποίηση, εμφανίζεται σχετική ιστοσελίδα που ενημερώνει ότι το email έχει επαληθευτεί.

Τεχνική Σημείωση:

Λόγω τεχνικών περιορισμών της υπηρεσίας Firebase, τα μηνύματα επαλήθευσης ενδέχεται να τοποθετούνται στο φάκελο *Spam* του παραλήπτη.



Εικόνα 4.2.3 α: Μήνυμα Επαλήθευσης Email

Digital Wallet ▾

Authentication

Users | Sign-in method | Templates | Usage | Settings | Extensions

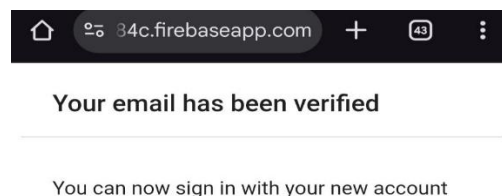
The following Authentication features will stop working when Firebase Dynamic Links shuts down soon: email link authentication for mobile apps, as well as Cordova OAuth support for web apps.

Search by email address, phone number, or user UID Add user ↻ ⋮

Identifier	Providers	Created ↓	Signed in	User UID
nikoskalamoutsos@gm...	✉	Nov 8, 2025	Nov 11, 2025	NuDfRnZycNlEEFO9l6pKgP5...

Rows per page: 50 1 - 1 of 1 < >

Εικόνα 4.2.3 b : Πίνακας Χρηστών Firebase (Email Authentication table)



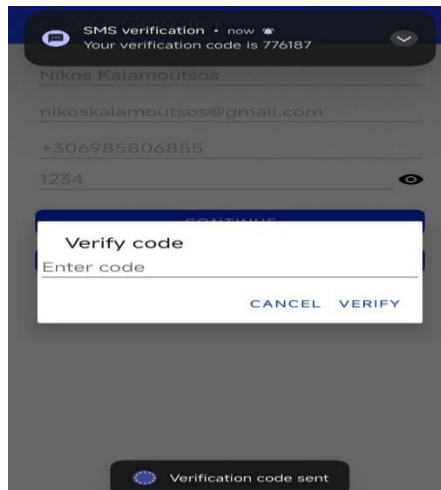
Εικόνα 4.2.4: Σελίδα Επιβεβαίωσης

Επαλήθευση μέσω SMS

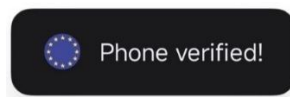
Με το Continue, αν έχει επαληθεύσει ο χρήστης το email και έχει δηλωθεί αριθμός τηλεφώνου, ο χρήστης λαμβάνει ειδοποίηση SMS με κωδικό επαλήθευσης. Ο κωδικός εισάγεται στο prompt **Verify Code**. Με επιτυχή

επαλήθευση, εμφανίζεται μήνυμα **Phone verified!** και η διαδικασία θεωρείται ολοκληρωμένη. Αν ο χρήστης πατήσει Cancel στο prompt, μπορεί να επαναλάβει τη διαδικασία αν πατήσει πάλι Continue, και θα λάβει νέα ειδοποίηση SMS με νέο κωδικό επαλήθευσης.

Τεχνική Σημείωση: Για λόγους ασφαλείας και οικονομικών περιορισμών (απουσία billing account στο Firebase SMS API), η επαλήθευση SMS υλοποιείται σε μορφή **mock verification**. Ο κωδικός παράγεται τοπικά και αποστέλλεται μέσω ειδοποίησης (Android Notification), χωρίς πραγματική SMS αποστολή, διασφαλίζοντας όμως την ίδια εμπειρία χρήσης.



Εικόνα 4.2.5: Λήψη Κωδικού SMS



Εικόνα 4.2.6: Επαλήθευση Τηλεφώνου

Ολοκλήρωση Εγγραφής

Μετά την ολοκλήρωση και των δύο επαληθεύσεων, ο χρήστης επιστρέφει αυτόματα στην οθόνη **LoginFragment**. Εμφανίζεται ειδοποίηση **“Registration successful! Please login.”**, που σηματοδοτεί την επιτυχή δημιουργία λογαριασμού.

Εικόνα 4.2.7 : Επιτυχής Ολοκλήρωση Εγγραφής και Μετάβαση στη Σύνδεση

Ενδεικτικοί Έλεγχοι Εισαγωγής Δεδομένων

- Το όνομα και το email είναι υποχρεωτικά πεδία.
- Το PIN πρέπει να αποτελείται από 4 έως 8 αριθμητικούς χαρακτήρες.
- Αν έχει δοθεί τηλέφωνο, η επαλήθευση μέσω SMS είναι υποχρεωτική.
- • Αν δεν έχει επαληθευτεί το email, ο λογαριασμός παραμένει ανενεργός.
- • Εμφανίζονται κατάλληλα μηνύματα σε περιπτώσεις αποτυχίας (π.χ. λάθος PIN, μη έγκυρος κωδικός SMS, μη επιβεβαιωμένο email).

Οι κανόνες ελέγχου και οι προειδοποιήσεις παρουσιάζονται αναλυτικά στην ενότητα **4.10**. Η εγγραφή πραγματοποιείται μόνο εφόσον όλα τα δεδομένα είναι έγκυρα και οι επαληθεύσεις ολοκληρωθούν επιτυχώς. Σε περίπτωση μη ολοκλήρωσης της εγγραφής, το προσωρινό Firebase authentication entry διαγράφεται αυτόματα.

4.3 Σύνδεση Χρήστη (LoginFragment)

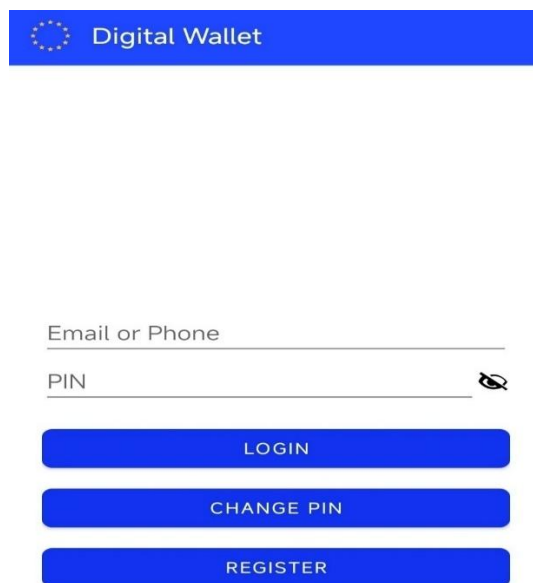
Περίπτωση Χρήσης: *Login*

Η οθόνη σύνδεσης παρέχει πρόσβαση με email ή τηλέφωνο και PIN. Η εφαρμογή εκτελεί τους παρακάτω ελέγχους:

1. Επαλήθευση ύπαρξης λογαριασμού στη βάση Room.
2. Σύγκριση του εισαγόμενου PIN με τον κατακερματισμένο κωδικό Bcrypt.

3. Καταγραφή αποτυχημένων προσπαθειών στον πίνακα **AuthLog** όταν το email ή το phone που έκανε input ο χρήστης υπάρχει στη βάση δεδομένων αλλά το pin είναι λάθος .

Μετά από τρεις αποτυχημένες προσπάθειες, ο λογαριασμός κλειδώνει προσωρινά για 15 λεπτά. Σε περίπτωση επιτυχούς ταυτοποίησης μέσω PIN, δημιουργείται νέα συνεδρία (SessionManager) και πραγματοποιείται μια ελαφριά σύνδεση στο Firebase (signInWithEmailAndPassword), για τη δημιουργία active Firebase session, το οποίο απαιτείται μόνο για την ορθή λειτουργία της διαγραφής λογαριασμού (Firebase current user deletion) και δεν συμμετέχει στη διαδικασία αυθεντικοποίησης του χρήστη . Στη συνέχεια, ο χρήστης μεταφέρεται στην οθόνη λίστας εγγράφων. Τα μηνύματα λαθών ή προειδοποιήσεων εμφανίζονται μέσω Snackbars ή Toasts, εξασφαλίζοντας άμεση ενημέρωση του χρήστη.



Εικόνα 4.3.1 : Οθόνη Σύνδεσης Χρήστη (LoginFragment)

4.4 Κεντρική Οθόνη Εγγράφων (DocumentsFragment)

Περίπτωση Χρήσης: *View Documents / Logout / Delete Profile*

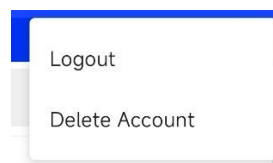
Η κύρια οθόνη εμφανίζει όλα τα έγγραφα του συνδεδεμένου χρήστη μέσω RecyclerView και DocumentsAdapter. Κάθε εγγραφή που εμφανίζεται σε αυτή την οθόνη περιλαμβάνει τον τύπο, τον εκδότη και την κατάσταση του εγγράφου. Επιλέγοντας ένα έγγραφο, ο χρήστης μεταφέρεται στο Document Detail Fragment, όπου μπορεί να δει περαιτέρω δεδομένα για το έγγραφο και επίσης να παρουσιάσει το QR code του εγγράφου για validation μέσω QR scanning.

Το Tool Bar Menu προσφέρει δυνατότητα διαγραφής λογαριασμού ή αποσύνδεσης. Το Floating Action Button κάτω δεξιά μεταφέρει τον χρήστη στο Add Document Fragment για να προσθέσει νέο έγγραφο .Η διαγραφή

λογαριασμού ζητά ρητή επιβεβαίωση πριν αφαιρεθούν οριστικά όλα τα δεδομένα του χρήστη από τη βάση.



Εικόνα 4.4.1 : Κεντρική Οθόνη Εγγράφων (DocumentsFragment)



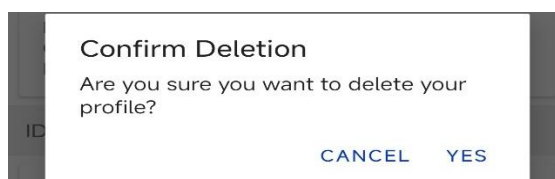
Εικόνα 4.4.2 : Tool Bar Menu (DocumentsFragment)

Περίπτωση Χρήσης: Logout

Η επιλογή "Logout" τερματίζει τη τωρινή συνεδρία (Session Manager), και ο χρήστης μεταφέρεται στο LoginFragment. Επίσης, η εφαρμογή αποσυνδέει τον χρήστη και από το **Firestore**, με αποτέλεσμα να μην θεωρείται πλέον *current Firestore user* στο σύστημα πιστοποίησης.

Περίπτωση Χρήσης: Delete Profile / Account

Η επιλογή "Delete Account" ανοίγει παράθυρο επιβεβαίωσης. Εφόσον ο χρήστης συμφωνήσει, η εφαρμογή προχωρά σε οριστική διαγραφή του λογαριασμού τόσο από τη Room βάση όσο και από το Firestore (διαγραφή του *current Firestore user*), καθώς και όλων των σχετιζόμενων εγγράφων μέσω του User Repository. Μετά την ολοκλήρωση της διαδικασίας, η συνεδρία τερματίζεται και ο χρήστης επιστρέφει στην οθόνη σύνδεσης, όπου εμφανίζεται το μήνυμα "Your profile has been deleted successfully."



Εικόνα 4.4.3 : Delete Profile Confirmation Prompt (DocumentsFragment)

4.5 Προσθήκη Νέου Εγγράφου (AddDocumentFragment)

Περίπτωση Χρήσης: *Add Document*

Επεκτάσεις: *Add ID Card, Add Driver's License, Add Ticket*

Η προσθήκη εγγράφου υλοποιείται μέσω φόρμας με δυναμικά πεδία ανάλογα με τον τύπο του εγγράφου. Ο χρήστης καταχωρεί εκδότη, ημερομηνίες έκδοσης/λήξης και πρόσθετες πληροφορίες (π.χ. Αριθμό Ταυτότητας, Ημερομηνία Γέννησης, Κατηγορία Άδειας, Θέση Εισιτηρίου).



The screenshot shows a mobile application interface for a 'Digital Wallet'. At the top is a blue header with a circular logo and the text 'Digital Wallet'. Below the header, the form is titled 'ID CARD' with a dropdown arrow. The form contains several input fields: 'Greece,Tripoli', '2016-10-31', and '2031-10-31'. Below these is a grey button labeled 'PICK QR IMAGE'. Underneath, it says 'QR loaded' above a placeholder image of a person. Below the image is a grey button labeled 'TAKE PHOTO'. The form continues with fields for 'Nikos Kalamoutsos', '2003-10-31', 'Nationality:' with a dropdown showing 'Austria', and '123456789 AN'. At the bottom is a large blue button labeled 'ADD DOCUMENT'.

Εικόνα 4.5.1 : Προσθήκη ID Card

Digital Wallet

DRIVER LICENSE

Greece,Tripoli

2016-10-31

2031-10-31

PICK QR IMAGE

QR loaded

TAKE PHOTO

Nikos Kalamoutsos

1998-10-31

123456789

B,BE,C1

ADD DOCUMENT

Εικόνα 4.5.2 : Προσθήκη Driver License

Digital Wallet

TICKET

Greece,Tripoli

2025-10-1

2025-11-30

PICK QR IMAGE

QR loaded

Concert

2025-11-30

123 B

OAKA

ADD DOCUMENT

Εικόνα 4.5.3 : Προσθήκη Ticket

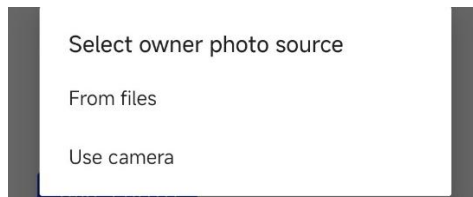
Η εφαρμογή επιτρέπει επιλογή ή λήψη φωτογραφίας και QR εικόνας από το σύστημα αρχείων ή την κάμερα.

Select QR source

From files

Use camera

Εικόνα 4.5.4 : Επιλογή πηγής QR Code



Εικόνα 4.5.5 : Επιλογή πηγής φωτογραφίας ιδιόκτητη για ID Card και Driver's license

Με το πάτημα του “Add document”, εμφανίζεται ένα prompt για να εισάγει και να επαληθεύσει ο χρήστης το pin του.

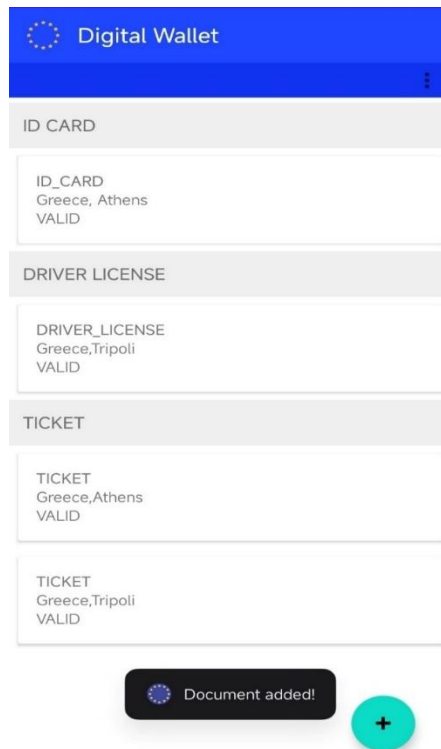


Εικόνα 4.5.6 : Επαλήθευση PIN

Γίνονται αυτόματα οι εξής έλεγχοι:

- Η ημερομηνία έκδοσης πρέπει να προηγείται της λήξης.
- Δεν επιτρέπεται διπλό QR ή Αριθμός Ταυτότητας ή Αριθμός Αδείας .
- Υποχρεωτική αποκωδικοποίηση QR πριν την εισαγωγή, με σαφές UI state (“QR loaded”) και αποκατάσταση σε περίπτωση σφάλματος.
- Υποχρεωτική φωτογραφία κατόχου για ID/Driver's License.
- Έλεγχος κατηγοριών διπλώματος έναντι λίστας επιτρεπτών τιμών.
- Έλεγχος αν η ημερομηνία γέννησης είναι στο μέλλον .
- Για δίπλωμα οδήγησης, ο κάτοχος πρέπει να είναι ≥ 18 ετών.
- Έλεγχος αν η ημερομηνία του Event είναι πριν τη σημερινή ημέρα

Σε επιτυχία προβάλλεται μήνυμα “*Document added!*” και η λίστα εγγράφων ενημερώνεται σε πραγματικό χρόνο.

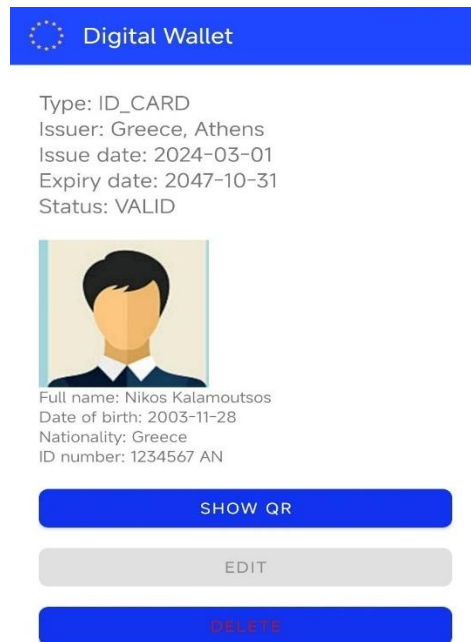


Εικόνα 4.5.7 : Επιτυχή εισαγωγή Document

4.6 Προβολή Εγγράφου (DocumentDetailFragment)

Περιπτώσεις Χρήσης: *View Document Details /Update expiry date /Show QR/ Delete document/Enter PIN*

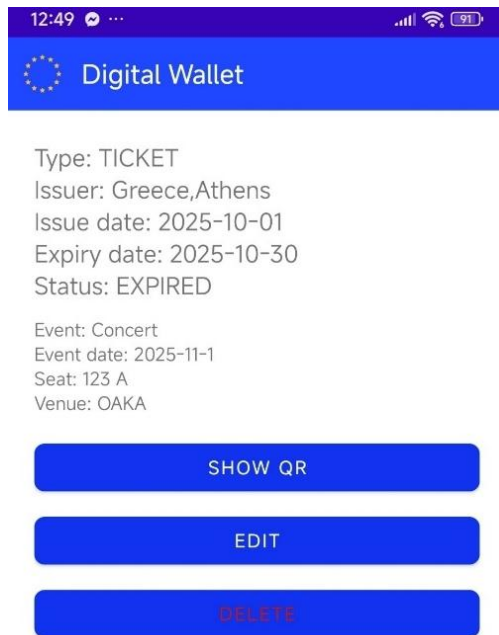
Η οθόνη λεπτομερειών εμφανίζει πλήρη στοιχεία του εγγράφου με δυναμική φόρμα που προσαρμόζεται στον τύπο του (ταυτότητα, δίπλωμα, εισιτήριο). Η φωτογραφία του κατόχου φορτώνεται μέσω URI, και αν λείπει εμφανίζεται προεπιλεγμένο εικονίδιο.



Εικόνα 4.6.1: ID CARD(DocumentDetailFragment)

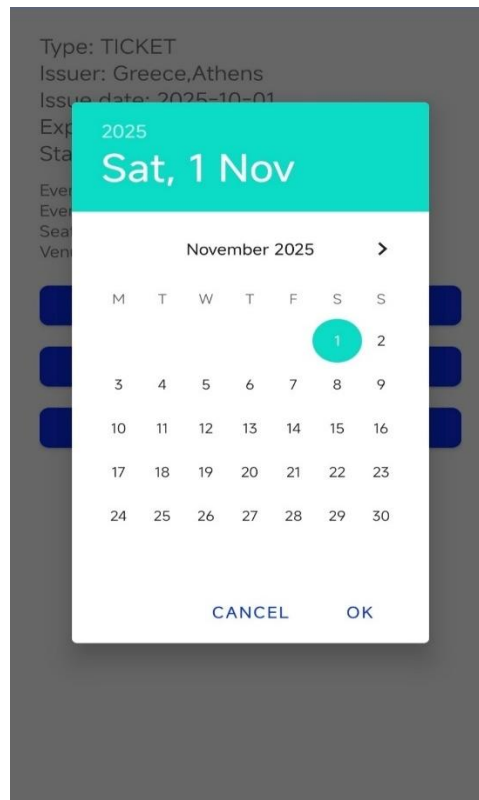


Εικόνα 4.6.2: Driver's License(DocumentDetailFragment)



Εικόνα 4.6.3: Ticket (DocumentDetailFragment)

Από την ίδια οθόνη ο χρήστης μπορεί να αλλάξει την ημερομηνία λήξης μέσω Date Picker, εφόσον η κατάσταση του εγγράφου είναι EXPIRED, αλλιώς το Edit button παραμένει disabled . Η νέα τιμή εγγράφεται μόνο αν είναι μεταγενέστερη της τρέχουσας και της ημερομηνίας έκδοσης. Η βάση ενημερώνεται ασύγχρονα και το UI ανανεώνεται αυτόματα μέσω ViewModel και LiveData.



Εικόνα 4.6.4: Date picker (αλλαγή ημερομηνίας λήξης)

Type: TICKET
Issuer: Greece,Athens
Issue date: 2025-10-01
Expiry date: 2025-11-01
Status: VALID

Event: Concert
Event date: 2025-11-1
Seat: 123 A
Venue: OAKA

SHOW QR

EDIT

DELETE



Expiry date updated.

Εικόνα 4.6.5 : Ενημέρωση Ημερομηνίας Λήξης

- Διαγράφει το έγγραφο από τη βάση δεδομένων.
- Εμφανίζει τον QR code του εγγράφου σε παράθυρο διαλόγου.

Η ενέργεια “Show QR” ανοίγει παράθυρο διαλόγου που προβάλλει τον κώδικα QR του εγγράφου. Ο κώδικας δημιουργείται δυναμικά με τη βιβλιοθήκη ZXing και προβάλλεται σε μορφή εικόνας ώστε να μπορεί να σαρωθεί από τρίτους φορείς για επαλήθευση.

Type: ID_CARD
Issuer: Greece, Athens
Issue date: 2024-03-01
Expiry date: 2047-10-31

QR



OK

Εικόνα 4.6.6 : Παρουσίαση QR Code (ShowQrDialogFragment)

Ολες αυτές οι ενέργειες απαιτούν την επαλήθευση του PIN του χρήστη



Εικόνα 4.6.7 : Επαλήθευση PIN

4.7 Αλλαγή PIN (ChangePinFragment)

Περίπτωση Χρήσης: Change PIN

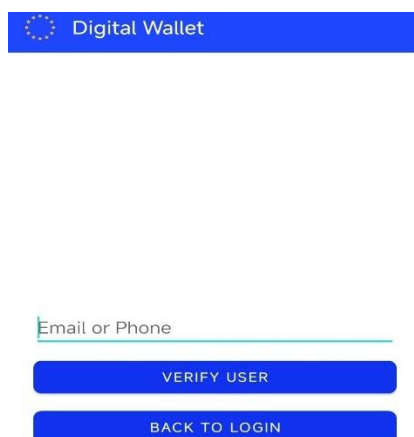
Επεκτάσεις: Verify Email / Verify Phone

Η λειτουργία αυτή επιτρέπει στον χρήστη να αλλάξει τον προσωπικό του κωδικό PIN, ο οποίος χρησιμοποιείται για την είσοδο στην εφαρμογή. Ακολουθεί την αρχή 'verify before modify', σύμφωνα με τις προδιαγραφές του EUDI Wallet για κρίσιμες ενέργειες χρήστη (credential management). Η διαδικασία περιλαμβάνει δύο στάδια: **(1) Επαλήθευση ταυτότητας** και **(2) Αλλαγή PIN**.

Η επαλήθευση πραγματοποιείται είτε μέσω email είτε μέσω τηλεφώνου, ανάλογα με τα στοιχεία του λογαριασμού.

4.7.1 Επαλήθευση Χρήστη

Αρχικά ο χρήστης εισάγει το email ή τον αριθμό τηλεφώνου του και επιλέγει VERIFY USER. Η εφαρμογή ελέγχει στη Room βάση αν υπάρχει καταχωρημένος λογαριασμός. Αν ναι, ενεργοποιείται η διαδικασία επαλήθευσης μέσω Firebase.



Εικόνα 4.7.1 : Οθόνη Επαλήθευσης Χρήστη

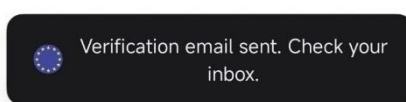
Επαλήθευση μέσω Email

Αν το αναγνωριστικό είναι email, αποστέλλεται μήνυμα Firebase που περιέχει σύνδεσμο επαλήθευσης (sign-in link). Ο χρήστης επικολλά τον σύνδεσμο στο παράθυρο διαλόγου **Paste verification link** για να ολοκληρωθεί η διαδικασία.

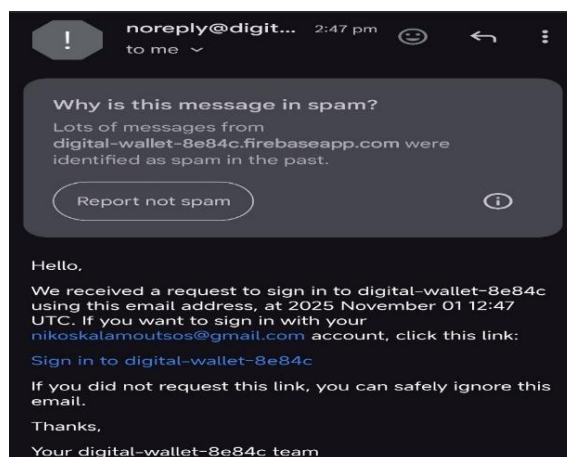
NIKOSKALAMOUTSOS@gmail.com

CONTINUE

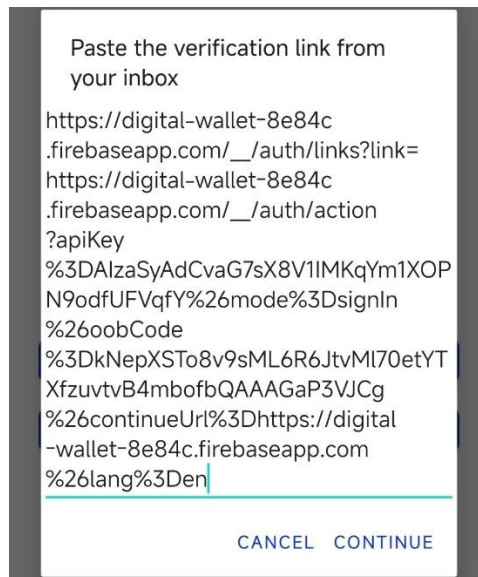
BACK TO LOGIN



Εικόνα 4.7.2 : Αποστολή Επαλήθευσης Email



Εικόνα 4.7.3 : Email Επαλήθευσης στο Gmail Inbox (ενδέχεται να σταλεί στο Spam)

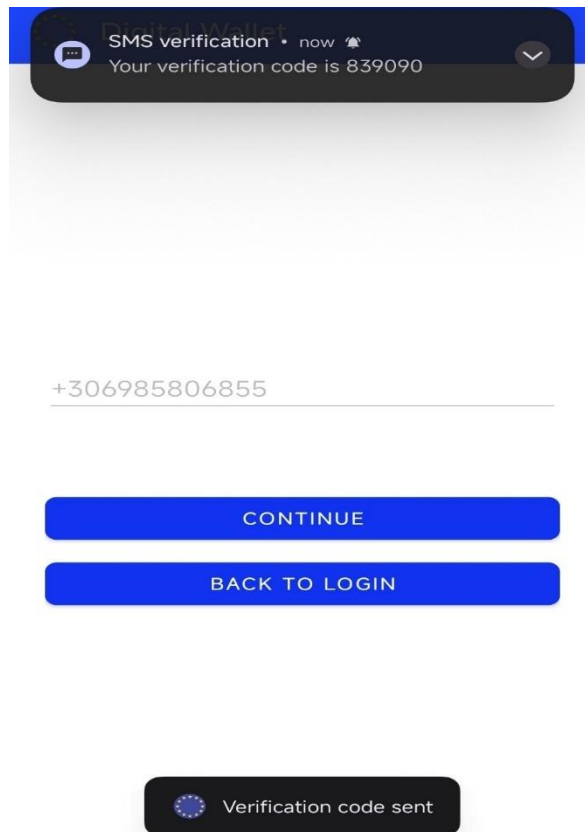


Εικόνα 4.7.4 : Παράθυρο Επικόλλησης Συνδέσμου Επαλήθευσης

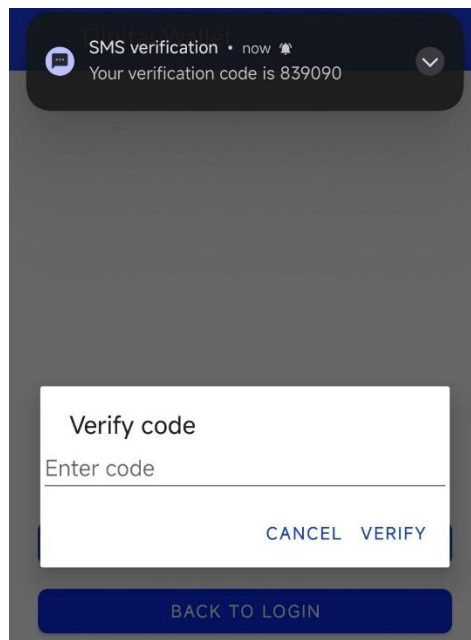
Τεχνική Σημείωση: Εξαιτίας περιορισμών του Firebase Auth, τα emails ενδέχεται να χαρακτηρίζονται ως *spam*. Παρ' όλα αυτά, ο σύνδεσμος παραμένει έγκυρος και η επαλήθευση ολοκληρώνεται κανονικά με το sign-in link.

Επαλήθευση μέσω SMS

Αν ο χρήστης επέλεξε επαλήθευση μέσω τηλεφώνου, η εφαρμογή δημιουργεί έναν **mock κωδικό SMS επαλήθευσης**, που εμφανίζεται ως ειδοποίηση στη συσκευή.



Εικόνα 4.7.5 : Αποστολή Mock Κωδικού Επαλήθευσης SMS



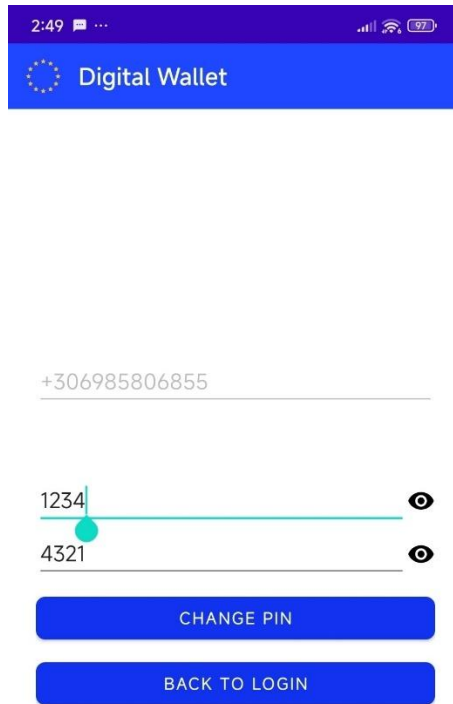
Εικόνα 4.7.6 : Παράθυρο Εισαγωγής Κωδικού Επαλήθευσης SMS

Τεχνική Σημείωση: Η αποστολή SMS είναι **mock διαδικασία**, καθώς δεν έχει προστεθεί billing account στο Firebase SMS API. Ο κωδικός παράγεται τοπικά και εμφανίζεται μέσω ειδοποίησης Android.

4.7.2 Αλλαγή PIN

Μετά την επιτυχή επαλήθευση ταυτότητας, ο χρήστης μεταβαίνει στη φόρμα αλλαγής PIN. Εκεί εισάγει το παλιό και το νέο PIN. Η εφαρμογή ελέγχει ότι:

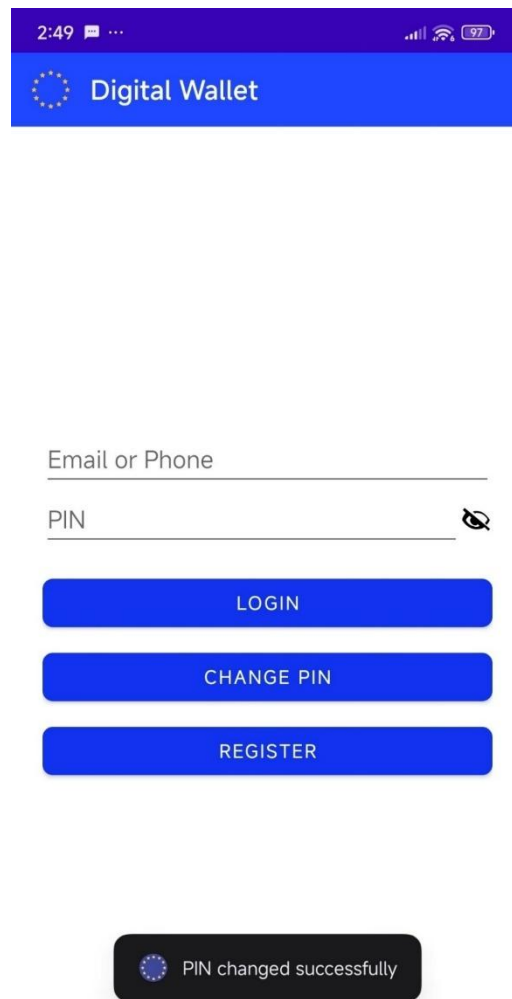
- το PIN περιέχει 4–8 ψηφία,
- είναι διαφορετικό από το παλιό,
- και ανήκει σε επαληθευμένο χρήστη.



The screenshot shows the 'Digital Wallet' app interface. At the top, there's a status bar with the time 2:49 and battery level 97%. Below the app title, the phone number +306985806855 is displayed. There are two input fields for PINs: the first contains '1234' and the second contains '4321'. Each field has a toggle icon to its right. Below the input fields are two blue buttons: 'CHANGE PIN' and 'BACK TO LOGIN'.

Εικόνα 4.7.7 : Οθόνη Εισαγωγής Παλαιού και Νέου PIN

Η εφαρμογή επιβεβαιώνει την επιτυχή αλλαγή μέσω μηνυματος toast και ενημερώνει τη βάση δεδομένων χρησιμοποιώντας **Bcrypt hashing** για την προστασία του νέου PIN.



Εικόνα 4.7.8 : Μήνυμα Επιτυχούς Αλλαγής PIN

4.8. Πολυγλωσσική Υποστήριξη

Η εφαρμογή υποστηρίζει έξι γλώσσες (Ελληνικά, Αγγλικά, Γερμανικά, Γαλλικά, Ιταλικά, Ισπανικά). Όλα τα μηνύματα, τίτλοι και ειδοποιήσεις προέρχονται από τα αντίστοιχα αρχεία `strings.xml`. Η επιλογή γλώσσας γίνεται αυτόματα με βάση τη ρύθμιση του συστήματος.

4:57

Ψηφιακό Πορτοφόλι

Email ή Τηλέφωνο

PIN

ΣΥΝΔΕΣΗ

ΑΛΛΑΓΗ PIN

ΕΓΓΡΑΦΗ

Εικόνα 4.8.1 : Login fragment (Greek)

4:57

Digitale Geldbörse

Name

Email

Telefon (optional)

PIN

REGISTRIEREN

ZURÜCK ZUR ANMELDUNG

Εικόνα 4.8.2 : Register fragment (German)

4:58

Digitale Geldbörse

nikoskalamoutsos@gmail.com

Alte PIN

Neue PIN

PIN ÄNDERN

ZURÜCK ZUR ANMELDUNG

Εικόνα 4.8.3 : Change pin (German)

4:59    97%

 Portafoglio Digitale

ID CARD 

Emittente :

Data di emissione :

Data di scadenza :

SELECCIONAR IMAGEN QR

Nessun QR selezionato



SCATTA FOTO

Nome completo :

Data di nascita :

Nazionalità :

Austria 

Numero di identificazione :

AGGIUNGI DOCUMENTO



Εικόνα 4.8.4 : Add document fragment (Italian)

5:01

Portefeuille Numérique

DRIVER LICENSE

Émetteur :

Date d'émission :

Date d'expiration :

SÉLECTIONNER UNE IMAGE QR

Aucun QR sélectionné

PRENDRE UNE PHOTO

Nom complet :

Date de naissance :

Numéro de permis :

Catégories :

AJOUTER UN DOCUMENT

Εικόνα 4.8.5 : Add document fragment (French)

The screenshot shows the 'Cartera Digital' app interface. At the top, there is a status bar with the time 5:02, signal strength, Wi-Fi, and battery level (76%). Below the status bar is a blue header with the app's logo and the text 'Cartera Digital'. The main content area is white and contains a form for adding a document fragment. The form starts with a dropdown menu labeled 'TICKET'. Below this are four text input fields: 'Emisor:', 'Fecha de emisión:', 'Fecha de vencimiento:', and 'Evento:'. There is a blue button labeled 'SELECCIONAR IMAGEN DE CÓDIGO QR' below the 'Fecha de vencimiento:' field. Below this button, the text 'No se ha seleccionado QR' is displayed. There are two more text input fields: 'Asiento:' and 'Lugar:'. At the bottom of the form is a blue button labeled 'AGREGAR DOCUMENTO'. The bottom of the screen shows the Android navigation bar with three icons: a square, a circle, and a triangle.

5:02 5G 76%

Cartera Digital

TICKET ▼

Emisor: _____

Fecha de emisión: _____

Fecha de vencimiento: _____

SELECCIONAR IMAGEN DE CÓDIGO QR

No se ha seleccionado QR

Evento: _____

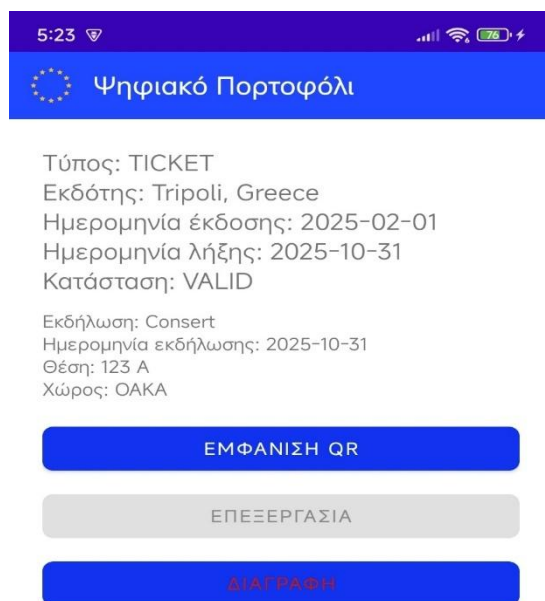
Fecha del evento: _____

Asiento: _____

Lugar: _____

AGREGAR DOCUMENTO

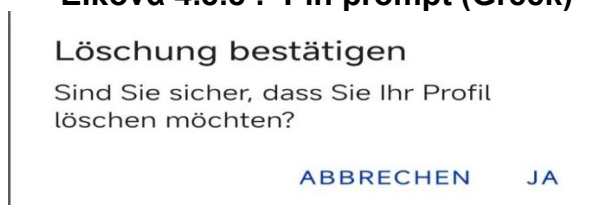
Εικόνα 4.8.6 : Add document fragment (Spanish)



Εικόνα 4.8.7 : Document details fragment (Greek)



Εικόνα 4.8.8 : Pin prompt (Greek)



Εικόνα 4.8.9 : Delete account prompt (German)

4.9 Συσχέτιση με το Διάγραμμα Περιπτώσεων Χρήσης

Πίνακας 4.9.1 : Συσχέτιση με το Διάγραμμα Περιπτώσεων Χρήσης

Fragment	Περίπτωση Χρήσης	Τύπος Σχέσης
RegisterFragment	Create profile	Basic
LoginFragment	Login to Wallet	Basic
DocumentsFragment	View Documents / Logout / Delete account	Basic
AddDocumentFragment	Add Document (extensions: Add ID card / Add driver's license / Add ticket)	Basic

Fragment	Περίπτωση Χρήσης	Τύπος Σχέσης
DocumentDetailFragment	View Document Details (extensions: Update expiry date, Delete document, Show QR)	Basic
ShowQrDialogFragment	Show Document QR Code	Extend
ChangePinFragment	Change PIN (includes : Verify identity , extensions: Verify Email/Phone number)	Basic

4.10 Προειδοποιήσεις και Μηνύματα Εφαρμογής (Warnings and Validation Messages)

Τα παρακάτω μηνύματα εμφανίζονται στην εφαρμογή μέσω Toast ή AlertDialog, καθοδηγώντας τον χρήστη στη σωστή χρήση και αποτρέποντας λανθασμένες ενέργειες ή ελλιπή εισαγωγή δεδομένων. Οι προειδοποιήσεις αυτές έχουν στόχο να εξασφαλίσουν την ορθή λειτουργία, την ακεραιότητα των δεδομένων και την ενημέρωση του χρήστη για κάθε σημαντική ενέργεια.

Πίνακας 4.10.1 : Γενικά Μηνύματα

Κατηγορία	Μήνυμα	Περιγραφή
Επιτυχία	<i>Document added successfully / PIN changed successfully / Registration successful!</i>	Εμφανίζεται μετά την επιτυχή ολοκλήρωση μιας ενέργειας.
Σφάλμα	<i>Something went wrong / Generic error</i>	Εμφανίζεται σε μη αναμενόμενα ή γενικά σφάλματα λειτουργίας.
Επαλήθευση	<i>PIN required / Invalid PIN</i>	Ζητείται PIN για ασφαλείς λειτουργίες (π.χ. διαγραφή, QR, ενημέρωση).

Πίνακας 4.10.2 : Οθόνη Σύνδεσης (LoginFragment)

Μήνυμα	Περιγραφή
<i>Invalid email or phone</i>	Μη έγκυρη μορφή email ή τηλεφώνου
<i>Invalid PIN</i>	Μη έγκυρη μορφή PIN
<i>Invalid credentials</i>	Εσφαλμένα στοιχεία σύνδεσης
<i>Account locked. Try again later</i>	Κλείδωμα λογαριασμού μετά από 3 αποτυχημένες προσπάθειες
<i>Something went wrong</i>	Γενικό σφάλμα επικοινωνίας ή βάσης.

Πίνακας 4.10.3 : Εγγραφή Χρήστη (RegisterFragment)

Μήνυμα	Περιγραφή
<i>All fields required</i>	Δεν έχουν συμπληρωθεί όλα τα απαραίτητα πεδία.
<i>Invalid email / Invalid phone</i>	Εσφαλμένη μορφή email ή τηλεφώνου.
<i>User already exists</i>	Ο χρήστης υπάρχει ήδη.
<i>This email is already in use</i>	Το Email χρησιμοποιείται από άλλον χρήστη.
<i>This phone number is already in use</i>	Το τηλεφωνο χρησιμοποιείται από άλλον χρήστη .
<i>Verification email sent</i>	Στάλθηκε email επαλήθευσης.
<i>Email not verified yet</i>	Ο χρήστης δεν έχει επαληθεύσει το email.
<i>Code sent / Invalid code / Phone verified</i>	Κατάσταση mock SMS επαλήθευσης.
<i>PIN must be 4–8 digits</i>	Το PIN δεν πληροί τους κανόνες μορφοποίησης.
<i>Registration successful! Please login.</i>	Ολοκληρωμένη εγγραφή, επιστροφή στη σύνδεση.

Η επαλήθευση τηλεφώνου πραγματοποιείται ως mock διαδικασία, λόγω απουσίας ενεργού Firebase SMS API.

Πίνακας 4.10.4 : Αλλαγή PIN (ChangePinFragment)

Μήνυμα	Περιγραφή
<i>Empty fields required</i>	Δεν έχουν εισαχθεί email ή τηλέφωνο για επαλήθευση.
<i>Invalid email or phone</i>	Μη έγκυρη είσοδος στοιχείων.
<i>Verification email sent</i>	Στάλθηκε σύνδεσμος επαλήθευσης email.
<i>Invalid code / Code sent</i>	Mock επαλήθευση τηλεφώνου.
<i>Tap Verify first</i>	Ο χρήστης δεν πάτησε Verify πριν συνεχίσει.
<i>PIN must be 4–8 digits</i>	PIN εκτός ορίων αποδεκτού μήκους.
<i>New PIN must be different from old PIN</i>	Το νέο PIN δεν πρέπει να είναι ίδιο με το παλιό.
<i>PIN changed successfully</i>	Η αλλαγή PIN ολοκληρώθηκε επιτυχώς.
<i>Something went wrong</i>	Γενικό σφάλμα αλλαγής.

Πίνακας 4.10.5 : Προσθήκη Εγγράφου (AddDocumentFragment)

Μήνυμα	Περιγραφή
<i>All fields required</i>	Απαιτούμενα πεδία δεν έχουν συμπληρωθεί.
<i>Select QR first</i>	Ο χρήστης δεν έχει επιλέξει QR εικόνα.

Μήνυμα	Περιγραφή
<i>Invalid DOB / Event date error</i>	Μη έγκυρες ημερομηνίες γέννησης ή εκδήλωσης.
<i>Driver must be at least 18 years old</i>	Οδηγός κάτω των 18 ετών.
<i>Invalid categories</i>	Μη έγκυρες κατηγορίες άδειας οδήγησης.
<i>Date error</i>	Η ημερομηνία έκδοσης προηγείται της λήξης.
<i>Duplicate QR / ID / License number</i>	Παραβίαση μοναδικών τιμών στη βάση.
<i>User not found</i>	Ο χρήστης δεν εντοπίστηκε.
<i>Document added successfully / Failed to add document</i>	Αποτέλεσμα προσθήκης εγγράφου.
<i>QR loaded / No QR found / Decode error</i>	Κατάσταση διαδικασίας ανάγνωσης QR.
<i>Permission required / Camera permission required</i>	Απόρριψη πρόσβασης κάμερας ή αρχείων.
<i>Photo capture failed</i>	Αποτυχία λήψης ή αποθήκευσης φωτογραφίας.

Η αποκωδικοποίηση QR βασίζεται στη βιβλιοθήκη ZXing και πραγματοποιείται τοπικά, χωρίς ανάγκη σύνδεσης στο διαδίκτυο.

Πίνακας 4.10.6 : Λεπτομέρειες Εγγράφου (DocumentDetailFragment)

Μήνυμα	Περιγραφή
<i>PIN required / Invalid PIN</i>	Απαραίτητη εισαγωγή PIN για ενημέρωση ή διαγραφή.
<i>User not found / Document not found</i>	Τα δεδομένα δεν υπάρχουν στη βάση.
<i>Expiry updated successfully</i>	Επιτυχής ενημέρωση ημερομηνίας λήξης.
<i>Expiry before issue / Expiry not later</i>	Λανθασμένες χρονικές τιμές λήξης.
<i>Update failed</i>	Αποτυχία ενημέρωσης εγγράφου.

Πίνακας 4.10.7 : Κεντρική Οθόνη Εγγράφων (DocumentsFragment)

Μήνυμα	Περιγραφή
<i>Profile deleted successfully</i>	Ο λογαριασμός διαγράφηκε επιτυχώς.
<i>Generic error</i>	Αποτυχία διαγραφής λογαριασμού.

Πίνακας 4.10.8 : Προβολή QR Εγγράφου (ShowQrDialogFragment)

Μήνυμα	Περιγραφή
<i>PIN required / Invalid PIN</i>	Ασφαλής προβολή QR απαιτεί PIN.
<i>Could not render QR image</i>	Αποτυχία δημιουργίας QR bitmap.
<i>Something went wrong</i>	Σφάλμα κατά τη δημιουργία ή φόρτωση QR.

5. ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΑΙ ΜΕΛΛΟΝΤΙΚΗ ΕΡΓΑΣΙΑ

5.1 Εισαγωγή

Το κεφάλαιο αυτό παρουσιάζει τα γενικά συμπεράσματα που προέκυψαν από την εκπόνηση της πτυχιακής εργασίας, συνοψίζοντας τα αποτελέσματα της ανάπτυξης, ανάλυσης και αξιολόγησης της εφαρμογής Digital Wallet. Η εργασία εντάσσεται στο πλαίσιο της ψηφιακής ταυτότητας και των ηλεκτρονικών πορτοφολιών της Ευρωπαϊκής Ένωσης και εστιάζει στην πρακτική διερεύνηση των αρχών που περιγράφονται στο European Digital Identity Wallet (EUDI Wallet) Reference Implementation.

Σκοπός ήταν η ανάπτυξη μιας εφαρμογής που να αποδεικνύει πώς οι αρχές του EUDI (ασφάλεια, προστασία ιδιωτικότητας, διαλειτουργικότητα και προσβασιμότητα) μπορούν να εφαρμοστούν με χρήση ανοιχτών τεχνολογιών Android, σε περιβάλλον ακαδημαϊκής έρευνας.

5.2 Συνοπτική Ανακεφαλαίωση της Υλοποίησης

Η εφαρμογή Digital Wallet σχεδιάστηκε και αναπτύχθηκε για λειτουργία σε συσκευές Android, χρησιμοποιώντας:

- Τη γλώσσα Kotlin,
- Το περιβάλλον ανάπτυξης Android Studio,
- Και τα Android Jetpack Components (Room, LiveData, ViewModel, Navigation, Data Binding).

Η αρχιτεκτονική **Model–View–ViewModel (MVVM)** επέτρεψε τον διαχωρισμό του UI από τη λογική των δεδομένων, τη βελτιωμένη επαναχρησιμοποίηση κώδικα και τη διασφάλιση ευκολίας συντήρησης. Ολόκληρη η δομή της εφαρμογής οργανώθηκε σε **πέντε επίπεδα**:

1. **UI Layer** – Διεπαφή και πλοήγηση μέσω Fragments.
2. **ViewModel Layer** – Διαχείριση κατάστασης και αλληλεπίδραση με το Repository.
3. **Repository Layer** – Επιχειρησιακή λογική, επικύρωση και σύνδεση με DAO.
4. **Data Layer (DAO)** – Εκτέλεση CRUD λειτουργιών στη βάση δεδομένων.
5. **Database Layer** – Ορισμός οντοτήτων και σχέσεων μέσω Room ORM.

Η σχεδίαση αυτή οδήγησε σε καθαρή, επεκτάσιμη και ανθεκτική αρχιτεκτονική, η οποία μπορεί να υποστηρίξει μελλοντικές προσθήκες, όπως πιστοποιητικά ταυτότητας (Verifiable Credentials) ή μηχανισμούς δικτύωσης (EUDI API Gateway).

5.3 Κύρια Επιτεύγματα και Αποτελέσματα

Η εργασία είχε ως βασικό στόχο να αποδείξει την εφικτότητα ανάπτυξης μιας λειτουργικής και ασφαλούς εφαρμογής ψηφιακού πορτοφολιού, προσαρμοσμένης στα ευρωπαϊκά πρότυπα.

Τα κύρια επιτεύγματα συνοψίζονται παρακάτω:

5.3.1 Αρχιτεκτονική και Τεχνολογική Πληρότητα

Η υλοποίηση αξιοποίησε σύγχρονες τεχνολογίες της Google (Jetpack, Room, Kotlin Coroutines) με γνώμονα τη σταθερότητα και την αποδοτικότητα. Η εφαρμογή αποδείχθηκε ανθεκτική στον κύκλο ζωής των Fragments και των Activities, με πλήρη απομόνωση της επιχειρησιακής λογικής από το UI.

5.3.2 Ασφάλεια και Αξιοπιστία

Η προστασία των δεδομένων αποτέλεσε πρωταρχικό στόχο. Η εφαρμογή ενσωμάτωσε:

- **Κατακερματισμό PIN** με χρήση Bcrypt.
- **Απομόνωση δεδομένων χρήστη** στη συσκευή (local-first design).
- **Καταγραφή συμβάντων (AuthLog)** για έλεγχο προσπαθειών σύνδεσης.
- **Μηχανισμό κλειδώματος λογαριασμού** μετά από τρεις αποτυχημένες προσπάθειες.

Αυτός ο συνδυασμός τεχνικών εξασφαλίζει τόσο την **εμπιστευτικότητα** όσο και την **ακεραιότητα** των δεδομένων, ακολουθώντας τις αρχές **Security by Design**.

5.3.3 Πολυγλωσσικότητα και Προσβασιμότητα

Η εφαρμογή μεταφράστηκε σε έξι γλώσσες (Ελληνικά, Αγγλικά, Γερμανικά, Ιταλικά, Γαλλικά, Ισπανικά), με πλήρη υποστήριξη από τον Translation Editor του Android Studio. Αυτό ενισχύει την προσβασιμότητα και την πολιτισμική ενσωμάτωση, στοιχεία ουσιώδη για μια πανευρωπαϊκή εφαρμογή όπως το EUDI Wallet.

5.3.4 Επεκτασιμότητα και Διαλειτουργικότητα

Η modular σχεδίαση (DAO – Repository – ViewModel) επιτρέπει την εύκολη επέκταση των λειτουργιών χωρίς επηρεασμό των υπάρχοντων ενοτήτων. Αυτό καθιστά δυνατή την προσθήκη νέων τύπων εγγράφων, μηχανισμών επαλήθευσης ή υπηρεσιών cloud στο μέλλον.

5.3.5 Ευχρηστία και Διεπαφή

Η διεπαφή χρήστη σχεδιάστηκε με γνώμονα την απλότητα και τη σαφήνεια. Τα μηνύματα, τα Toasts και τα dialogs παρέχουν άμεση πληροφόρηση για

επιτυχείς ή αποτυχημένες ενέργειες, υποστηρίζοντας το πρότυπο User-Centric Design.

5.4 Συμπεράσματα

Η ανάπτυξη του Digital Wallet απέδειξε ότι είναι εφικτή η δημιουργία μιας τοπικά λειτουργούσας, ασφαλούς και ευέλικτης εφαρμογής ψηφιακής ταυτότητας που ακολουθεί τις ευρωπαϊκές προδιαγραφές.

Βασικά συμπεράσματα:

1. **Η αρχιτεκτονική MVVM** εξασφαλίζει στιβαρή δομή και επεκτασιμότητα.
2. **Η χρήση Room ORM** επιτρέπει αποτελεσματική και ασφαλή διαχείριση δεδομένων χωρίς ανάγκη εξωτερικού server.
3. **Η προσέγγιση Privacy by Design** εγγυάται ότι ο έλεγχος των δεδομένων ανήκει αποκλειστικά στον χρήστη.
4. **Η πολυγλωσσική υποστήριξη** καθιστά την εφαρμογή έτοιμη για πανευρωπαϊκή χρήση.
5. **Η τοπική υλοποίηση** μπορεί να αποτελέσει πρότυπο για μεγαλύτερα συστήματα ψηφιακής ταυτότητας.

Η εργασία συνεισφέρει στην κατανόηση της πρακτικής πλευράς του EUDI Reference Implementation, αποδεικνύοντας ότι οι αρχές του μπορούν να εφαρμοστούν σε πραγματικό περιβάλλον Android, με ελάχιστες απαιτήσεις υποδομής.

Η συνολική αξιολόγηση δείχνει ότι η εφαρμογή:

- Λειτουργεί αξιόπιστα,
- Είναι φιλική προς τον χρήστη,
- Και παρέχει υψηλό επίπεδο ασφάλειας για το επίπεδο πολυπλοκότητας της υλοποίησης.

5.5 Μελλοντική Εργασία

Αν και η εφαρμογή ολοκληρώθηκε επιτυχώς, υπάρχουν αρκετές δυνατότητες βελτίωσης και επέκτασης που θα μπορούσαν να τη φέρουν πιο κοντά στις πλήρεις προδιαγραφές του EUDI Wallet.

5.5.1 Υποστήριξη Verifiable Credentials

Η μελλοντική ενσωμάτωση μηχανισμών OpenID4VCI (Verifiable Credential Issuance) και OpenID4VP (Verifiable Presentation) θα επιτρέψει την ανταλλαγή ψηφιακών διαπιστευτηρίων μεταξύ εφαρμογών και φορέων [11]. Έτσι, το Digital Wallet θα μπορεί να λειτουργήσει ως πραγματικό EUDI wallet, επαληθεύσιμο μέσω trust frameworks [13].

5.5.2 Trust Frameworks και Ευρωπαϊκές Λίστες Εμπιστοσύνης

Η ενσωμάτωση των **European Trust Lists (ETL)** θα επιτρέψει τον αυτόματο έλεγχο εγκυρότητας των εκδοτών εγγράφων και των πιστοποιητικών υπογραφής, προσφέροντας επιπλέον ασφάλεια και αξιοπιστία.

5.5.3 Προηγμένοι Μηχανισμοί Ασφάλειας

Η αποθήκευση ευαίσθητων δεδομένων σε Android Keystore ή Secure Element θα ενίσχυε σημαντικά την ασφάλεια. Επιπλέον, η προσθήκη βιομετρικής αυθεντικοποίησης (δακτυλικό αποτύπωμα, αναγνώριση προσώπου) θα επέτρεπε μια πιο φυσική και ασφαλή εμπειρία χρήσης.

5.5.4 Ενσωμάτωση Cloud Services με Κρυπτογράφηση

Αν και η εφαρμογή ακολουθεί local-first σχεδίαση, μπορεί να υποστηριχθεί προαιρετικός συγχρονισμός μέσω end-to-end κρυπτογραφημένων cloud υπηρεσιών, ώστε ο χρήστης να ανακτή τα δεδομένα του σε πολλαπλές συσκευές.

5.5.5 Πιστοποίηση και Συμμόρφωση με EUDI Sandbox

Η προσαρμογή της εφαρμογής για συμμετοχή στο **EUDI Sandbox** θα προσέφερε τη δυνατότητα επίσημης αξιολόγησης βάσει των ευρωπαϊκών κριτηρίων.

Αυτό θα καθιστούσε την υλοποίηση πιστοποιήσιμη και εν δυνάμει εντάξιμη σε πιλοτικά προγράμματα.

5.5.6 Ανάλυση Χρηστικότητας και Απόκρισης

Μελλοντικές εκδόσεις θα μπορούσαν να συμπεριλάβουν:

- Δοκιμές χρηστικότητας με πραγματικούς χρήστες (UX testing).
- Μέτρηση απόδοσης (latency, memory footprint).
- Συμμόρφωση με οδηγίες **WCAG 2.1** για άτομα με αναπηρίες.

5.5.7 Πραγματική Επαλήθευση μέσω SMS

Στην τρέχουσα υλοποίηση, η επαλήθευση μέσω SMS πραγματοποιείται με τη μορφή *mock verification*, λόγω της απουσίας ενεργού Firebase billing account για χρήση του SMS API. Ο κωδικός επαλήθευσης παράγεται τοπικά και αποστέλλεται στον χρήστη μέσω ειδοποίησης (Android Notification), προσομοιώνοντας τη διαδικασία αποστολής SMS.

Σε μελλοντική έκδοση, θα μπορούσε να ενσωματωθεί πλήρης μηχανισμός επαλήθευσης τηλεφώνου μέσω Firebase SMS Verification ή εναλλακτικά μέσω υπηρεσιών τρίτων παρόχων (π.χ. Twilio, Vonage, AWS SNS). Η υλοποίηση αυτή θα περιλαμβάνει:

- Αποστολή πραγματικού SMS με μοναδικό verification code.
- Καταχώρηση του τηλεφώνου στη βάση Room, ώστε να συνδέεται άμεσα με το προφίλ του χρήστη, όπως ήδη γίνεται για το email.
- Επιβεβαίωση του κωδικού μέσω Firebase Auth ή αντίστοιχης υπηρεσίας.
- Μηχανισμό αυτόματης λήψης SMS (automatic retrieval) για βελτίωση UX.
- Ενημέρωση του πεδίου “phoneVerified” στη Room database, εξασφαλίζοντας την ίδια αξιοπιστία με την επαλήθευση email.

Η ενσωμάτωση πλήρους SMS verification θα αυξήσει τη συνολική ασφάλεια του Digital Wallet και θα επιτρέψει τη χρήση του για πιο αυθεντικές διαδικασίες ταυτοποίησης, σε πλήρη εναρμόνιση με τις ευρωπαϊκές προδιαγραφές του EUDI Wallet για multi-factor authentication (MFA).

5.6 Συνολική Εκτίμηση και Συνεισφορά

Η εργασία αυτή συνδυάζει θεωρητική έρευνα και πρακτική ανάπτυξη, παρέχοντας μια ολιστική προσέγγιση στο θέμα της ψηφιακής ταυτότητας. Το Digital Wallet αποδεικνύει ότι με σωστή αρχιτεκτονική και σύγχρονα εργαλεία, είναι δυνατή η δημιουργία ασφαλών, προσβάσιμων και επεκτάσιμων εφαρμογών, συμβατών με το ευρωπαϊκό πλαίσιο.

Η συνεισφορά της εργασίας συνοψίζεται ως εξής:

- [1] Παρουσιάζει μια **πλήρως λειτουργική τοπική υλοποίηση** ψηφιακού πορτοφολιού.
- [2] Εφαρμόζει τις αρχές **Privacy by Design** και **Security by Design**.
- [3] Αναδεικνύει τη σημασία της **πολυγλωσσικής διεπαφής** για την ευρωπαϊκή διαλειτουργικότητα.
- [4] Θέτει τις βάσεις για περαιτέρω ερευνητική ανάπτυξη σε τομείς ταυτοποίησης, πιστοποίησης και διαχείρισης διαπιστευτηρίων.

Η εργασία επιβεβαιώνει ότι η τεχνολογική υλοποίηση μπορεί να συνδυαστεί με την πολιτική και κοινωνική διάσταση της ψηφιακής ταυτότητας, στηρίζοντας το όραμα για μια ενιαία, ασφαλή και φιλική προς τον χρήστη ευρωπαϊκή ψηφιακή εμπειρία.

ΠΙΝΑΚΑΣ ΟΡΟΛΟΓΙΑΣ

Ξενόγλωσσος Όρος	Ελληνικός Όρος
Digital Wallet	Ψηφιακό Πορτοφόλι
Repository Pattern	Πρότυπο Αποθετηρίου
Data Access Object (DAO)	Αντικείμενο Πρόσβασης Δεδομένων
Room Database	Βάση Δεδομένων Room
ViewModel	Μοντέλο Προβολής
LiveData	Ζωντανά Δεδομένα
QR Code	Κωδικός QR
Firebase	Πλατφόρμα Υπηρεσιών Firebase
Android Jetpack	Πακέτο Βιβλιοθηκών Android Jetpack
Session Manager	Διαχειριστής Συνεδρίας
Navigation Graph	Διάγραμμα Πλοήγησης
Fragment	Τμήμα Διεπαφής (Fragment)
User Interface (UI)	Διεπαφή Χρήστη
Verifiable Credential	Επαληθεύσιμο Διαπιστευτήριο

ΣΥΝΤΜΗΣΕΙΣ – ΑΡΚΤΙΚΟΛΕΞΑ – ΑΚΡΩΝΥΜΙΑ

Συντομογραφία	Επεξήγηση
MVVM	Model – View – ViewModel
DAO	Data Access Object
UI	User Interface
PIN	Personal Identification Number
QR	Quick Response
DB	Database
SDK	Software Development Kit
XML	eXtensible Markup Language
EUDI	European Digital Identity
API	Application Programming Interface

ΠΑΡΑΡΤΗΜΑ Ι – ΚΩΔΙΚΑΣ ΕΦΑΡΜΟΓΗΣ

1. Εισαγωγή

Το παράρτημα αυτό περιλαμβάνει επιλεγμένα αποσπάσματα κώδικα της εφαρμογής Digital Wallet, τα οποία τεκμηριώνουν τη δομή και λειτουργία του συστήματος που αναπτύχθηκε στο πλαίσιο της παρούσας πτυχιακής εργασίας. Η ανάπτυξη ακολουθεί την αρχιτεκτονική **MVVM (Model–View–ViewModel)** και βασίζεται σε βιβλιοθήκες του Android Jetpack, όπως τα Room, ViewModel, LiveData και Navigation.

2. Επίπεδο Παρουσίασης (UI Elements – Fragments)

Η εφαρμογή διαθέτει πολλαπλά Fragments για την αλληλεπίδραση του χρήστη:

- **LoginFragment** – Είσοδος με email ή τηλέφωνο και PIN.
- **RegisterFragment** – Δημιουργία λογαριασμού και επιβεβαίωση στοιχείων μέσω Firebase (email ή mock SMS).
- **DocumentsFragment** – Προβολή αποθηκευμένων εγγράφων, αποσύνδεση, και διαγραφή προφίλ.
- **AddDocumentFragment** – Προσθήκη νέου εγγράφου μέσω QR και επιβεβαίωσης PIN.
- **DocumentDetailFragment** – Προβολή στοιχείων, ενημέρωση λήξης και διαγραφή εγγράφων.
- **ChangePinFragment** – Αλλαγή PIN με έλεγχο ταυτότητας.
- **ShowQrDialogFragment** – Προβολή QR για επαλήθευση.

Παράδειγμα:

```
val toolbar=binding.topAppBar
toolbar.inflateMenu(R.menu.documents_menu)
toolbar.setOnMenuItemClickListener { item ->
    when (item.itemId) {
        R.id.action_logout -> {
            FirebaseAuth.getInstance().signOut()
            SessionManager.clearSession(requireContext())
            findNavController().navigate(R.id.action_documentsFragment_to_loginFragment)
            true
        }
        R.id.action_delete_user -> {
            android.app.AlertDialog.Builder(requireContext())
                .setTitle(getString(R.string.confirm_deletion_title))
                .setMessage(getString(R.string.confirm_deletion_message))
                .setPositiveButton(getString(R.string.yes)) { _, _ ->
                    if(userId!=null)
                        userViewModel.deleteUser(userId)
                }
                .setNegativeButton(getString(R.string.cancel), null)
                .show()
            true
        }
    }
    else -> false
}
```

```
}  
}
```

3. Επίπεδο ViewModel

Τα ViewModels συνδέουν τα UI στοιχεία με το Repository, διασφαλίζοντας ανεξαρτησία και καθαρή αρχιτεκτονική.

AuthViewModel

```
fun login(emailOrPhone: String, pin: String, ip: String? = null) {  
    viewModelScope.launch {  
        loginResult.value = userRepository.login(emailOrPhone, pin, ip)  
    }  
}
```

DocumentViewModel

```
fun loadUserDocuments(userId: Int) {  
    viewModelScope.launch {  
        repo.updateDocumentValidity(userId)  
        documents.value = repo.getDocumentsByUser(userId)  
    }  
}
```

4. Επίπεδο Repository

Τα Repositories περιλαμβάνουν την επιχειρησιακή λογική και τους ελέγχους εγκυρότητας.

UserRepository

```
suspend fun login(emailOrPhone: String, pin: String, ip: String? = null): Result<User> {  
  
    val user = findByEmailOrPhone(emailOrPhone)  
    ?: return Result.failure(Exception("INVALID_CREDENTIALS"))  
  
    val fifteenMinsAgo = System.currentTimeMillis() - 15 * 60 * 1000  
    val failCount = db.authLogDao().countRecentFailedAttempts(user.userId, fifteenMinsAgo)  
    if (failCount >= 3) return Result.failure(Exception("ACCOUNT_LOCKED"))  
  
    val result = BCrypt.verifier().verify(pin.toCharArray(), user.pinHash)  
    val status = if (result.verified) AuthStatus.SUCCESS else AuthStatus.FAILED  
    db.authLogDao().insert(AuthLog(userId = user.userId, status = status, ipAddress = ip))  
  
    return if (result.verified) Result.success(user) else  
    Result.failure(Exception("INVALID_CREDENTIALS"))  
}
```

DocumentRepository

```
suspend fun getDocumentQrCode(userId: Int, documentId: Int, pin: String): Result<String> {  
    val user =  
        db.userDao().getById(userId) ?: return Result.failure(Exception("USER_NOT_FOUND"))  
    val ok = BCrypt.verifier().verify(pin.toCharArray(), user.pinHash).verified  
    if (!ok) return Result.failure(Exception("INVALID_PIN"))  
    val doc = db.documentDao().getDocument(documentId, userId)  
    return if (doc != null) Result.success(doc.qrCode)  
}
```

```

    } else Result.failure(Exception("DOC_NOT_FOUND"))
}

```

5. Επίπεδο Βάσης Δεδομένων (Room)

User.kt

```

@Entity(
    tableName = "users",
    indices = [Index(value = ["email"], unique = true), Index(value = ["phone"], unique = true)]
)
data class User(
    @PrimaryKey(autoGenerate = true) val userId: Int = 0,
    val name: String,
    val email: String,
    val phone: String?,
    val pinHash: String,
    val createdAt: Long = System.currentTimeMillis()
)

```

Document.kt

```

@Entity(
    tableName = "documents",
    foreignKeys = [
        ForeignKey(
            entity = User::class,
            parentColumns = ["userId"],
            childColumns = ["userId"],
            onDelete = ForeignKey.CASCADE
        )
    ],
    indices = [Index(value = ["userId"], unique = true), Index(value = ["qrCode"], unique = true)]
)
data class Document(
    @PrimaryKey(autoGenerate = true) val documentId: Int = 0,
    val userId: Int,
    val type: DocumentType,
    val issuer: String,
    val issueDate: Long,
    val expiryDate: Long,
    val qrCode: String,
    val status: DocumentStatus = DocumentStatus.VALID,
    val createdAt: Long = System.currentTimeMillis()
)

```

DocumentDetails.kt

```

@Entity(
    tableName = "id_card_details",
    foreignKeys = [
        ForeignKey(
            entity = Document::class,
            parentColumns = ["documentId"],
            childColumns = ["documentId"],
            onDelete = ForeignKey.CASCADE,
            onUpdate = ForeignKey.NO_ACTION
        )
    ]
)

```

```

    ],
    indices = [Index(value = ["documentId"], unique =
true),Index(value=["idNumber"],unique=true)]
)
data class IdCardDetails(
    @PrimaryKey val documentId: Int,    // FK to documents.documentId
    val fullName: String? = null,
    val nationality: String? = null,
    val idNumber: String? = null,
    val dateOfBirth: String? = null,
    val ownerPhotoUri: String? = null
)

@Entity(
    tableName = "driver_license_details",
    foreignKeys = [
        ForeignKey(
            entity = Document::class,
            parentColumns = ["documentId"],
            childColumns = ["documentId"],
            onDelete = ForeignKey.CASCADE,
            onUpdate = ForeignKey.NO_ACTION
        )
    ],
    indices = [Index(value = ["documentId"], unique =
true),Index(value=["licenseNumber"],unique=true)]
)
data class DriverLicenseDetails(
    @PrimaryKey val documentId: Int,    // FK to documents.documentId
    val fullName: String? = null,
    val dateOfBirth: String? = null,
    val licenseNumber: String? = null,
    val categories: String? = null,
    val ownerPhotoUri: String? = null
)

@Entity(
    tableName = "ticket_details",
    foreignKeys = [
        ForeignKey(
            entity = Document::class,
            parentColumns = ["documentId"],
            childColumns = ["documentId"],
            onDelete = ForeignKey.CASCADE,
            onUpdate = ForeignKey.NO_ACTION
        )
    ],
    indices = [Index(value = ["documentId"], unique = true)]
)
data class TicketDetails(
    @PrimaryKey val documentId: Int,    // FK to documents.documentId
    val eventName: String? = null,
    val eventDate: String? = null,
    val seat: String? = null,
    val venue: String? = null
)

```

UserDao.kt

```

@Dao
interface UserDao {

```

```

@Query("SELECT * FROM users WHERE email = :email LIMIT 1")
suspend fun findByEmail(email: String): User?

@Query("SELECT * FROM users WHERE phone = :phone LIMIT 1")
suspend fun findByPhone(phone: String): User?

@Insert(onConflict = OnConflictStrategy.ABORT)
suspend fun insert(user: User): Long

@Delete
suspend fun delete(user: User): Int

@Query("UPDATE users SET pinHash = :pinHash WHERE userId = :userId")
suspend fun updatePin(userId: Int, pinHash: String): Int

@Query("SELECT * FROM users WHERE userId = :userId")
suspend fun getById(userId: Int): User?
}

```

DocumentsDao.kt

```

interface DocumentDao {

    // Existing
    @Query("SELECT * FROM documents WHERE userId = :userId")
    suspend fun getDocumentsByUser(userId: Int): List<Document>

    @Insert(onConflict = OnConflictStrategy.ABORT)
    suspend fun insert(document: Document): Long

    @Delete
    suspend fun delete(document: Document)

    @Query("DELETE FROM documents WHERE documentId = :docId")
    suspend fun deleteById(docId: Int)

    @Query("SELECT * FROM documents WHERE documentId = :docId AND userId = :userId LIMIT 1")
    suspend fun getDocument(docId: Int, userId: Int): Document?
    .
    .
    .

    @Upsert
    suspend fun upsertId(details: IdCardDetails)

    @Upsert
    suspend fun upsertDriver(details: DriverLicenseDetails)

    @Upsert
    suspend fun upsertTicket(details: TicketDetails)

    // New detail getters
    @Query("SELECT * FROM id_card_details WHERE documentId = :docId")
    suspend fun getIdDetails(docId: Int): IdCardDetails?

    @Query("SELECT * FROM driver_license_details WHERE documentId = :docId")
    suspend fun getDriverDetails(docId: Int): DriverLicenseDetails?

    @Query("SELECT * FROM ticket_details WHERE documentId = :docId")

```



```

suspend fun getTicketDetails(docId: Int): TicketDetails?
}

```

6. Υποστηρικτικές Κλάσεις

SessionManager

```

object SessionManager {
    private const val PREF_NAME = "user_session"
    private const val KEY_USER_ID = "user_id"

    fun saveUserId(context: Context, userId: Int) {
        val prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        prefs.edit { putInt(KEY_USER_ID, userId) }
    }

    fun getUserId(context: Context): Int? {
        val prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        val id = prefs.getInt(KEY_USER_ID, -1)
        return if (id != -1) id else null
    }

    fun clearSession(context: Context) {
        val prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
        prefs.edit { clear() }
    }
}

```

DatabaseProvider

```

object DatabaseProvider {
    @Volatile
    private var INSTANCE: AppDatabase? = null

    private val MIGRATION_1_2 = object : Migration(1, 2) {
        override fun migrate(db: SupportSQLiteDatabase) {
            db.execSQL("ALTER TABLE id_card_details ADD COLUMN ownerPhotoUri TEXT")
            db.execSQL("ALTER TABLE driver_license_details ADD COLUMN ownerPhotoUri TEXT")
        }
    }
    //other migrations
    .
    .
    .

    fun getDatabase(context: Context): AppDatabase =
        INSTANCE ?: synchronized(this) {
            INSTANCE ?: Room.databaseBuilder(
                context.applicationContext,
                AppDatabase::class.java,
                "digitalwalletdb"
            )
                .addMigrations(MIGRATION_1_2, MIGRATION_2_3, MIGRATION_3_4,
MIGRATION_4_5)
                .fallbackToDestructiveMigrationFrom()
                .build()
                .also { INSTANCE = it }
        }
}

```

```
}  
}
```

7. Adapter

DocumentsAdapter

```
sealed class DocumentListItem {  
    data class Header(val type: DocumentType) : DocumentListItem()  
    data class Item(val document: Document) : DocumentListItem()  
}  
  
class DocumentsAdapter(  
    private var items: List<DocumentListItem>,  
    private val onDocumentClick: (Document) -> Unit  
) : RecyclerView.Adapter<RecyclerView.ViewHolder> {  
    .  
    .  
}
```

Ο adapter γεφυρώνει τη λίστα εγγράφων της βάσης με το RecyclerView της οθόνης DocumentsFragment.

ΠΑΡΑΡΤΗΜΑ II – ΔΙΑΓΡΑΜΜΑ ΠΛΟΗΓΗΣΗΣ & ΠΟΛΥΓΛΩΣΣΙΑ

1. Διάγραμμα Πλοήγησης (Navigation Graph)

Εικόνα Β.1 – Διάγραμμα Πλοήγησης Εφαρμογής

Το παρακάτω διάγραμμα παρουσιάζει τη συνολική ροή και τις συνδέσεις ανάμεσα στα κύρια Fragments της εφαρμογής Digital Wallet, όπως ορίζονται στο αρχείο nav_graph.xml.

Η χρήση του **Android Jetpack Navigation Component** επιτρέπει:

- ασφαλή πλοήγηση και μεταφορά δεδομένων μέσω Safe Args,
- αυτόματη διαχείριση του back stack,
- σαφή και επεκτάσιμη αρχιτεκτονική πλοήγησης εντός της εφαρμογής.



Διάγραμμα παραρτήματος: fragment navigation graph

2. Πολυγλωσσική Υποστήριξη

Η εφαρμογή **Digital Wallet** υποστηρίζει έξι ευρωπαϊκές γλώσσες, σύμφωνα με την ευρωπαϊκή προδιαγραφή του EUDI Wallet:

- GR **Ελληνικά (el)**
- GB **Αγγλικά (en)**
- DE **Γερμανικά (de)**
- IT **Ιταλικά (it)**
- ES **Ισπανικά (es)**
- FR **Γαλλικά (fr)**

Οι μεταφράσεις υλοποιήθηκαν μέσω του συστήματος Android string resources (strings.xml) και διαχειρίζονται από τον Translation Editor του Android Studio. Όλα τα μηνύματα, τίτλοι και ειδοποιήσεις προσαρμόζονται αυτόματα στη γλώσσα που έχει ορίσει ο χρήστης στο λειτουργικό του Android.

2.1 Ενδεικτικές Συμβολοσειρές (Translations Example)

```
<string name="login">Σύνδεση / Login / Anmelden / Accedi / Iniciar sesión / Se  
connecter</string>  
<string name="register">Εγγραφή / Register / Registrieren / Registrati /  
Registrarse / S'inscrire</string>  
<string name="logout">Αποσύνδεση / Logout / Abmelden / Disconnetti / Cerrar  
sesión / Déconnexion</string>
```

Οι αντίστοιχες οθόνες σε κάθε γλώσσα παρουσιάζονται αναλυτικά στο **Κεφάλαιο 4 – Ενότητα 4.8 (Πολυγλωσσική Υποστήριξη)**, με ενδεικτικά screenshots του UI σε όλες τις διαθέσιμες γλώσσες.

2.2 Πίνακας Κρατών-Μελών Ε.Ε. (Απόσπασμα)

```
<string-array name="eu_countries">  
  <item>Ελλάδα / Greece / Griechenland</item>  
  <item>Γερμανία / Germany / Deutschland</item>  
  <item>Ιταλία / Italy / Italien</item>  
  <item>Ισπανία / Spain / Spanien</item>  
  <item>Γαλλία / France / Frankreich</item>  
  ...  
</string-array>
```

ΒΙΒΛΙΟΓΡΑΦΙΚΕΣ ΑΝΑΦΟΡΕΣ

Βιβλία

- [1] Pressman, R. S. & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. 9th Edition. McGraw-Hill Education.
- [2] Eriksson, H.-E., Penker, M., Lyons, B., & Fado, D. (2004). *UML 2 Toolkit*. John Wiley & Sons.
- [3] Κυπριανός, Χ. (2019). *Να ένα Android – Οδηγός ανάπτυξης εφαρμογών Android*. Εκδόσεις Κλειδάριθμος.
- [4] Sommerville, I. (2016). *Software Engineering*. 10th Edition. Pearson Education.
- [5] Βασιλακόπουλος, Γ. (2018). *Πληροφοριακά Συστήματα Προσεγγίσεις Ανάπτυξη Πραγμάτωση*. Εκδόσεις: Τσότρας

Ηλεκτρονικές Πηγές

- [6] European Commission. (2024). *European Digital Identity Wallet – Reference Implementation*. [Online]. Available at: <https://github.com/eu-digital-identity-wallet/.github/blob/main/profile/reference-implementation.md>
- [7] Google Developers. (2024). *Guide to App Architecture (Android Jetpack)*. [Online]. Available at: <https://developer.android.com/topic/architecture>
- [8] JetBrains. (2024). *Kotlin Documentation*. [Online]. Available at: <https://kotlinlang.org/docs/home.html>
- [9] Firebase Documentation. (2024). *Authenticate with Firebase using Email and Password on Android*. [Online]. Available at: <https://firebase.google.com/docs/auth/android/password-auth>
- [10] Android Developers. (2024). *Room Persistence Library Documentation*. [Online]. Available at: <https://developer.android.com/training/data-storage/room>
- [11] OpenID Foundation. (2024). *OpenID for Verifiable Credential Issuance (OpenID4VCI) Specification*. [Online]. Available at: https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html

Επιστημονικές Πηγές και Έρευνες

- [12] European Union Agency for Cybersecurity (ENISA). (2023). *Guidelines for Secure Digital Identity Wallets*. Publications Office of the European Union.
- [13] W3C Consortium. (2022). *Verifiable Credentials Data Model 2.0*. [Online]. Available at: <https://www.w3.org/TR/vc-data-model-2.0/>